Laporan Tugas Besar III
IF2211 - Strategi Algoritma

Implementasi *Pattern Matching* pada *Chromium Browser Extension* untuk Deteksi Judi Online



Dipersiapkan oleh:

Aufa Rienaldifaza Ahmad	13524027
Yavie Azka Putra Araly	13524077
Muhammad Dzaky Atha F.	18223124

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
JL. GANESHA 10, BANDUNG 40132
2026

DAFTAR ISI

DAFTAR ISI **2**
BAB I **4**
DESKRIPSI TUGAS **4**
BAB II **7**
LANDASAN TEORI **7**
2.1. Permasalahan String Matching 7
2.2. Knuth-Morris-Pratt 7
2.3. Boyer-Moore 8
2.4. Weighted Levenshtein Distance 9
2.5. Aho-Corasick 11
2.6. Rabin Karp 13
2.7. Ekspresi Regular 13
2.8. Browser Extension 15
BAB III **17**
ANALISIS PEMECAHAN MASALAH **17**
3.1. Langkah-Langkah Pemecahan Masalah 17
3.2. Proses Pemetaan Masalah Menjadi Elemen-Elemen Algoritma Pattern Matching .. 18
3.3. Fitur Fungsional dan Arsitektur Extension yang Dibangun 21
3.4. Contoh Ilustrasi Kasus 22
BAB IV **24**
IMPLEMENTASI DAN PENGUJIAN **24**
4.1. Pseudocode Algoritma 24
4.1.1. Knuth-Morris-Pratt 24
4.1.2. Boyer-Moore 26
4.1.3. Regular Expression 27
4.1.4. Fuzzy Matching 29
4.1.5. Aho-Corasick 31
4.1.6. Rabin-Karp 33
4.2. Spesifikasi Teknis Program 34
4.2.1. Struktur Proyek 34
4.2.2. Struktur Data 38
4.2.3. Fungsi dan Prosedur 41
4.3. Tata Cara Penggunaan Extension 45
4.4. Hasil dan Analisis Pengujian 51
4.4.1. Exact Matching dengan KMP / BM / Aho-Corasick 51

4.4.2. Regex + Rabin-Karp (pola <kata><angka>)..... 51
4.4.3. Fuzzy Matching Substitusi Visual..... 52
4.4.4. Keyword di Dalam Tabel..... 53
4.4.5. Fuzzy Matching Unicode Homoglyph..... 53
4.4.6. Konteks Kalimat Realistis..... 54
4.4.7. Edge Case: Keyword muncul berkali-kali dalam satu kalimat..... 54
4.4.8. Edge Case: Keyword sebagai Substring dari Kata Lain..... 54
4.4.9. Edge Case: Case Sensitivity..... 55
4.4.10. Edge Case: Teks Panjang dengan Keyword tersebar..... 55
4.4.11. Edge Case: Keyword di Elemen Nested..... 56
4.4.12. Edge Case: Keyword di Elemen Interaktif dan Struktural..... 56
4.4.13. Edge Case: Teks Kosong, Whitespace, dan Node Trivial..... 56
4.4.14. Edge Case: Keyword di Antara Tanda Baca..... 57
4.4.15. Edge Case: Keyword Overlapping dan Adjacent..... 57
4.4.16. Edge Case: Kombinasi Fuzzy + Tanda Baca..... 58
BAB V..... 59
KESIMPULAN DAN SARAN..... 59
5.1. Kesimpulan..... 59
5.2. Saran..... 59
LAMPIRAN..... 61
Tautan Repository Github:..... 61
Surat Pernyataan..... 61
REFERENSI..... 64

BAB I
DESKRIPSI TUGAS

Pada tugas besar ini, buatlah sebuah ***chromium browser extension*** yang dapat mendeteksi informasi yang mengandung unsur judol. Sebagai catatan, seperti yang biasa kita temukan dalam website sekarang dengan format <kata><angka>. Contoh: GACOR99, MAXWIN88, MADU308, dan sebagainya. Sistem harus memiliki **minimal fitur** dengan detail seperti berikut.

1. Extension yang dibuat pada tugas besar ini dikembangkan dengan bahasa **Typescript**. Dalam proses pencocokan kata kunci, sistem wajib mengimplementasikan algoritma **Knuth-Morris-Pratt** dan **Boyer Moore** serta **RegEx**. Untuk algoritma selain RegEx, pencarian dilakukan menggunakan file keyword.txt (file ini boleh dimodifikasi sesuai kebutuhan) yang dipisahkan dengan endline untuk melakukan string matching secara iterative, sehingga pengguna tidak memasukkan input, tapi sistem yang membaca file tersebut secara langsung.

   Untuk pencarian dengan regex, format pencarian **seminimal mungkin** dapat menghandle 2 atau 3 angka di belakang kata tanpa spasi. Contoh: MAXWIN234, SLOT99. Test case yang diberikan nanti akan sangat bervariasi, sehingga pastikan pencarian RegEx diimplementasikan dengan detail dan dapat **menghandle banyak edge case**. *Adapun juga algoritma Aho-Corasick dan Rabin Karp apabila ingin mengerjakan bonus, sesuai tertera di Spesifikasi Bonus dibawah.*
2. Perlu diketahui bahwa, Developer Judol pun juga membuat penamaan dari websitenya, seperti pada teknik *Phising,* jadi membuat nama domain yang serupa dengan pada umumnya, namun nyatanya berbeda. Ini terjadi oleh karena adanya perbedaan kode komputer (*unicode*), dan komputer mengiranya adalah karakter yang berbeda. Contoh:

Gacor999	Gɑcor999
HOKI88	H0KI88

dan lain sebagainya

Maka dari itu, sistem harus bisa membaca kecocokan secara exact matching dan juga fuzzy matching (apabila tidak ditemukan kecocokan yang persis). Untuk setiap kata kunci yang tidak memiliki kemunculan sama sekali pada saat *exact matching*, lakukan pencarian kembali dengan perhitungan tingkat kemiripan menggunakan algoritma Weighted Levenshtein Distance. Penyesuaian bobot (*weight*) ini krusial karena pelaku sering kali menyiasati strategi dengan menggunakan huruf pengganti yang mirip secara visual (misalnya mengubah huruf 'O' menjadi angka '0', 'A' menjadi '4', atau 'I' menjadi '1'). Implementasi harus memperhatikan hal-hal berikut:

1. Weighted Cost: Berikan bobot penalti yang lebih kecil untuk substitusi karakter yang mirip secara visual dibandingkan dengan karakter yang benar-benar berbeda.
2. Thresholding: Tentukan nilai ambang batas (*threshold*) kemiripan yang tepat untuk memastikan sistem tetap memberikan hasil yang relevan namun tetap menghindari munculnya *false positive*.

Algoritma ini memungkinkan sistem untuk tetap menampilkan hasil pencarian yang relevan meskipun terdapat perbedaan minor, kesalahan ketik, atau upaya manipulasi teks oleh pengguna. *Perlu diperhatikan bahwa secara konsep, exact matching juga merupakan bagian dari fuzzy matching*.

$$\operatorname{lev}(a,b) = \begin{cases} |a| & \text{if } |b| = 0, \\ |b| & \text{if } |a| = 0, \\ \operatorname{lev}\left(\operatorname{tail}(a), \operatorname{tail}(b)\right) & \text{if } \operatorname{head}(a) = \operatorname{head}(b), \\ 1 + \min \begin{cases} \operatorname{lev}\left(\operatorname{tail}(a), b\right) \\ \operatorname{lev}\left(a, \operatorname{tail}(b)\right) \\ \operatorname{lev}\left(\operatorname{tail}(a), \operatorname{tail}(b)\right) \end{cases} & \text{otherwise} \end{cases}$$

Weighted Levenshtein Distance Formula

3. Jadi sistem menjalan pengecekan dengan hirarki sebagai berikut:

a. Exact Matching: Algoritma *pattern matching* untuk mencari kata kunci dasar dari keyword.txt
b. RegEx Matching: Secara paralel atau sekuensial, jalankan Regex untuk menangkap pola <kata><angka>
c. Fuzzy Matching: Kalau string dalam DOM tidak lolos namun memiliki kemiripan, jalankan dengan Weighted Levenshtein Distance dalam upaya manipulasi karakter

4. Apabila ditemukan result keyword, maka elemen DOM terkait harus ditandai, dengan warna bebas. Namun highlight tidak boleh merusak layout, tepat sasaran dan dapat terhapus saat rescanning.
5. Saat user hover pada elemen yang terdeteksi, maka tampilkan tooltip dengan informasi berikut:
    a. Keyword yang terdeteksi
    b. Algoritma yang digunakan
    c. Jumlah kemunculan
    d. Waktu eksekusi

    Tooltip dibuat dengan custom DOM element.

BAB II
LANDASAN TEORI

2.1. Permasalahan String Matching

Suatu permasalahan *string matching* secara formal dapat didefinisikan dengan pencarian *pattern* P di dalam sembarang *string* T dengan

1. T merupakan *text* dengan panjang n karakter
2. P merupakan *pattern* yang memiliki panjang m dengan asumsi $m << n$ yang akan dicari di dalam T.

2.2. Knuth-Morris-Pratt

Algoritma Knuth-Morris-Pratt (KMP) merupakan algoritma pencocokan string yang melakukan pencarian dari kiri ke kanan seperti brute force dengan menggunakan informasi dari proses pencocokan sebelumnya untuk menghindari perbandingan yang repetitif. Sebelum proses pencarian dimulai, KMP melakukan preprocessing terhadap pattern untuk membangun fungsi pinggiran. Fungsi pinggiran $b(k)$ didefinisikan sebagai panjang *prefix* terpanjang dari $P[1..k]$ yang juga merupakan suffix dari $P[1..k]$. Fungsi ini memungkinkan algoritma untuk mengetahui seberapa jauh pattern dapat digeser ketika terjadi ketidakcocokan *(mismatch)* tanpa mengulang perbandingan yang sudah diketahui cocok. Contohnya, diberikan sebuah *string* "abaaba", akan ditemukan bahwa $b(k) = 2$ melalui proses komputasi pada tabel 2.1.

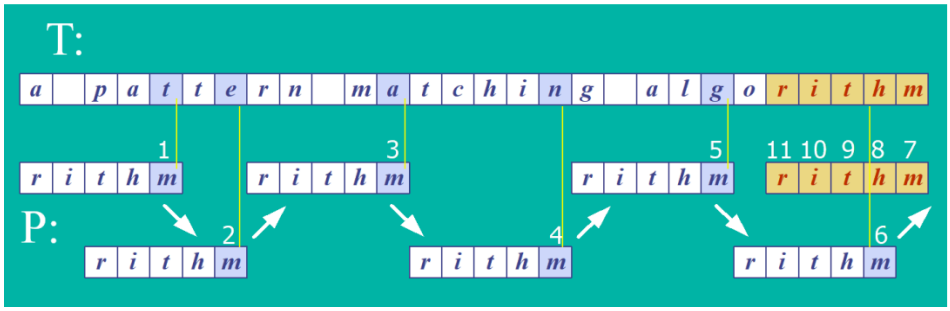
Tabel 2.1 Pemrosesan *Bounding Function* untunk String "abaaba"

j	0	1	2	3	4	5
P[j]	a	b	a	a	b	a
k	0	1	2	3	4	-
b(k)	0	0	1	1	2	-

Setelah *bounding function* ditemukan, akan dilakukan proses pencarian pada suatu *string* sembarang. Ketika terjadi *mismatch* pada posisi P[j] dengan T[i], pointer teks i tidak mundur, tetapip pointer pattern j diperbarui menjadi b[j-1] sehingga bagian prefix pattern yang sudah diketahui cocok tidak perlu dibandingkan ulang.

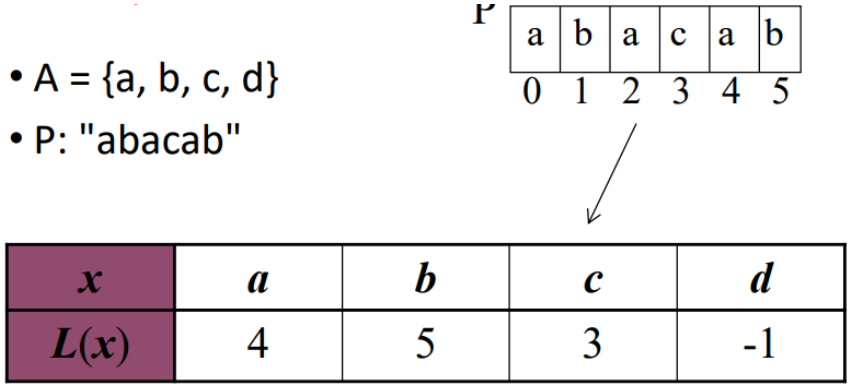
2.3. Boyer-Moore

Algoritma Boyer-Moore (BM) melakukan pencarian dari kanan ke kiri pada suatu pattern dan menggunakan strategi *looking-glass technique* atau *character-jump technique*. Pada *looking-glass technique*, kita mencari P di dalam T dengan bergerak mundur melalui P. Pada *character-jump technique*, *mismatch* ditemukan jika pada kasus T[i] == x, *pattern* P[j] tidak sama dengan T[i].



Gambar 1.2.1. Contoh *String Matching* dengan algoritma Boyer-Moore

Algoritma Boyer-Moore menggunakan *Last Occurence Function*, yaitu *preprocessing* terhadap *pattern* P dan sebuah alfabet A dengan $L(x)$ terdefinisi sebagai indeks i terbesear yang memenuhi $P[i] == x$ atau -1 jika tidak ditemukan indeksnya.



x	*a*	*b*	*c*	*d*
L(x)	4	5	3	-1

Gambar 1.2.2. Contoh Pembangunan Tabel *Last Occurence*

Setelah *last occurence* dibangun dan disimpan, misalnya dalam suatu *hashmap*. Dilakukan langkah pencarian berdasarkan kasus terjadinya *mismatch* pada $T[s + j] = x$ dan $P[j]$,

- Kasus 1: Jika x ada di *pattern* dan posisi kemunculan terakhirnya $L(x) < j$, geser pattern ke kanan sebesar $j - L(x)$.
- Kasus 2: Jika x ada di pattern, tetapi $L(x) >= j$, geser *pattern* 1 posisi ke kanan.
- Kasus 3: Jika x tidak ada dalam *pattern*, geser *pattern* sehingga $P[0]$ sejajar dengan $T[s + j + 1]$.

2.4. Weighted Levenshtein Distance

Levenshtein Distance klasik mengukur jumlah minimum operasi edit untuk mengubah string s1 menjadi s2. Namun untuk kasus deteksi judol, penyebar konten sering memanipulasi keyword dengan mengganti huruf dengan karakter yang mirip secara visual, misalnya SL0T untuk SLOT, atau M4XWIN untuk MAXWIN. Substitusi seperti ini secara teknis adalah "kesalahan", tapi secara visual hampir tidak terlihat.

Levenshtein standar akan menghitung SL0T → SLOT sebagai jarak 1 (satu substitusi), sama dengan SLOT → XYZW yang jelas-jelas berbeda. Versi weighted mengatasi ini dengan memberikan bobot berbeda berdasarkan seberapa mirip dua karakter secara visual. Terdapat beberapa operasi edit dasar:

Tabel 2.2 Operasi Edit Dasar Algoritma Levenshtein

Operasi	Contoh	Bobot
Insert	SLOT → SLOTS	1.0
Delete	SLOTS → SLOT	1.0
Substitusi Normal	SLOT → XLOT	1.0
Substitusi Visual	SL0T → SLOT (0→O)	0.2
Substitusi Identik	S → S	0.0

Bobot 0.2 untuk substitusi visual dipilih karena cukup kecil untuk tidak mendominasi jarak total, namun cukup besar untuk tetap dibedakan dari karakter yang identik. Selain itu, developer judi online juga sering menggunakan karakter Unicode Homoglyph. Karakter ini tetap dapat dibaca manusia, namun berbeda di mata komputer. Berikut karakter Unicode Homoglyph yang ditranslasikan pada program ini:

Tabel 2.3 Karakter Unicode Homoglyph yang Ditranslasikan pada Program

Latin	Karakter Pengganti	Asal
A	α, A, a, A, ɑ	Greek, Cycrillic
E	e, E, ε	Greek, Cycrillic
O	o, O, o, O	Greek, Cycrillic
M	μ, M, м, M	Greek, Cycrillic
H	H	Greek
S	s, S	Cycrillic
X	x, X	Cycrillic

Levenshtein Distance menggunakan Dynamic Programming untuk menentukan jarak antara S1 dengan S2. Fungsi rekurensnya adalah sebagai berikut:

$$d(i,j) = \begin{cases} i & \text{jika } j = 0 \\ j & \text{jika } i = 0 \\ \min \begin{cases} d(i-1,\ j) + 1 & \text{(delete)} \\ d(i,\ j-1) + 1 & \text{(insert)} \\ d(i-1,\ j-1) + w(s_1[i],\ s_2[j]) & \text{(substitusi)} \end{cases} & \text{jika } i,j > 0 \end{cases}$$

Hasil Levenshtein kemudian dinormalisasi ke rentang [0.0, 1.0] agar bisa dibandingkan antar pasangan string dengan panjang berbeda:

$$\text{similarity}(s_1, s_2) = 1 - \frac{\text{levenshtein}(s_1, s_2)}{\max(|s_1|, |s_2|)}$$

Token dianggap match terhadap keyword jika similarity ≥ 0.75. Threshold ini dipilih berdasarkan pertimbangan berikut:

- Kata 4 karakter dengan 1 substitusi normal: similarity ≈ 0.75 (tepat di batas)
- Kata 4 karakter dengan 1 substitusi visual (bobot 0.2): similarity ≈ 0.95 (jauh di atas batas)
- Kata yang berbeda sepenuhnya: similarity ≈ 0.0–0.4

Alur Fuzzy dalam sistem:

1. Tokenisasi flatTextMap menggunakan regex /[a-zA-Z0-9\u0080-\uFFFF]+/g untuk memecah pada tanda baca sambil mempertahankan karakter unicode
2. Skip token yang panjangnya < 4 karakter (terlalu pendek, rawan false positive) atau > 20 karakter (hampir pasti bukan keyword judol)
3. Skip token yang range posisinya sudah dicakup exact matching — tidak perlu fuzzy jika sudah terdeteksi
4. Untuk setiap token, bandingkan dengan semua keyword; skip jika selisih panjang > 2 karakter
5. Ambil keyword dengan similarity tertinggi per token sebagai hasil

Algoritma ini memiliki kompleksitas waktu $O(T \times K \times L^2)$, di mana T = jumlah token, K = jumlah keyword, L = panjang rata-rata string dan kompleksitas ruang $O(m \times n)$ untuk tabel DP per pasangan.

2.5. Aho-Corasick

KMP dan BM dirancang untuk mencari satu pattern dalam teks. Untuk mencari k keyword, keduanya harus dijalankan k kali secara terpisah, menghasilkan kompleksitas $O(n \times k)$. Aho-Corasick mengatasi ini dengan membangun sebuah finite state automaton dari semua keyword sekaligus, sehingga seluruh teks hanya perlu di-scan satu kali. Automaton Aho-Corasick terdiri dari tiga komponen:

1. Trie: semua keyword dimasukkan ke dalam trie (pohon prefiks). Setiap node merepresentasikan sebuah prefix dari satu atau lebih keyword.
2. Failure links: untuk setiap node u, failure link menunjuk ke node v yang merepresentasikan suffix terpanjang dari string di u yang juga merupakan prefix dari keyword lain. Konsep ini identik dengan failure function di KMP, hanya diperluas ke struktur trie.
3. Output links: shortcut ke node terdekat dalam failure chain yang merupakan akhir dari suatu keyword. Diperlukan untuk mendeteksi keyword yang merupakan suffix dari keyword lain

Pada langkah awal algoritma, kita perlu membangun Trie:

1. Untuk setiap keyword, telusuri trie dari root karakter per karakter. Buat node baru jika belum ada. Tandai node akhir sebagai output node.
2. Hitung Failure Links via BFS:

a. Semua anak langsung root: failure link → root
b. Untuk node u yang dicapai via karakter c dari node v:

$$\text{fail}(u) = \delta(\text{fail}(v), c)$$

di mana δ(node, c) berarti mengikuti edge c dari node, atau ikuti failure link jika tidak ada edge tersebut, sampai root.

3. Hitung Output Links:

$$\text{outputLink}(u) = \begin{cases} \text{fail}(u), & \text{jika fail}(u) \text{ adalah output node} \\ \text{outputLink}(\text{fail}(u)), & \text{selainnya} \end{cases}$$

Setelah Trie dibangun, kita akan melakukan pencarian. Scan teks dari kiri ke kanan, pertahankan pointer curr ke node automaton saat ini. Misal, dengan daftar keyword {SLOT, TOGEL, SLOT99} dan string yang dicek adalah DAFTARSLOT99SEKARANG, langkahnya adalah sebagai berikut:

- Automaton berjalan normal sampai menemukan SLOT di posisi 6
- Di posisi yang sama, output link mendeteksi bahwa SLOT adalah bagian dari SLOT99
- Saat 99 selesai dibaca, SLOT99 juga dilaporkan sebagai match

Kompleksitas algoritma Aho-Corasick adalah O(Σ × M + n + z), dimana Σ adalah jumlah alfabet, M = panjang total semua keyword, dan z adalah jumlah total kemunculan. Sedikit tambahan, karena mata kuliah ini berada di lab yang sama dengan mata kuliah Teori Bahasa Formal Dan Otomata (TBFO), Aho-Corasick dapat dinyatakan secara formal sebagai sebuah Deterministic Finite Automaton (DFA):

$$M = (Q,\ \Sigma,\ \delta,\ q_0,\ F)$$

Di mana

Q : himpunan semua node dalam trie yang dibangun dari keyword $\{p_1, p_2, \ldots, p_k\}$

Σ : himpunan karakter yang mungkin muncul dalam teks

$\delta : Q \times \Sigma \to Q$: fungsi transisi

$q_0 \in Q$: root node (state awal)

$F \subseteq Q$: himpunan node yang merupakan akhir dari minimal satu keyword (state penerima)

Dengan fungsi transisi:

$$\delta(u,\ c) = \begin{cases} \text{child}(u,\ c) & \text{jika edge } c \text{ ada dari node } u \\ \delta(\text{fail}(u),\ c) & \text{jika edge } c \text{ tidak ada dan } u \neq q_0 \\ q_0 & \text{jika edge } c \text{ tidak ada dan } u = q_0 \end{cases}$$

2.6. Rabin Karp

Rabin-Karp menggunakan pendekatan yang berbeda dari semua algoritma di atas. Alih-alih membandingkan karakter satu per satu, algoritma ini membandingkan nilai hash dari window teks dengan hash pattern. Inovasi utamanya adalah rolling hash yang memungkinkan nilai hash diperbarui dalam O(1) saat window bergeser. Hash polinomial untuk string s dengan panjang m memiliki formula berikut:

$$\text{hash}(s) = \left(s[0] \cdot B^{m-1} + s[1] \cdot B^{m-2} + \cdots + s[m-1] \cdot B^{0}\right) \bmod P$$

Dengan B = 31 sebagai basis dan P = 1.000.000.007 sebagai modulus (bilangan prima besar untuk meminimalkan collision). Saat window bergeser dari posisi i ke i+1, karakter s[i] keluar dan karakter s[i+m] masuk:

$$\text{hash_baru} = \left((\text{hash_lama} - s[i] \cdot B^{m-1}) \times B + s[i+m]\right) \bmod P$$

Nilai B^{m-1} dihitung sekali di awal menggunakan fast exponentiation (O(log m)), kemudian digunakan di setiap langkah pergeseran. Catatan implementasi: hasil intermediate bisa negatif karena operasi mod dengan bilangan yang dikurangi. Oleh karena itu selalu tambahkan P sebelum mod:

$$\text{hash} = ((\text{hash} \bmod P) + P) \bmod P$$

Ketika hash_window == hash_pattern, ada dua kemungkinan:

1. True match: substring memang cocok dengan pattern
2. Spurious hit: hash collision, substring sebenarnya berbeda

Oleh karena itu, setiap kali hash cocok, dilakukan verifikasi karakter per karakter untuk memastikan match yang sesungguhnya. Probabilitas spurious hit ≈ 1/P — dengan P = 10^9, ini sangat kecil secara praktis. Algoritma ini memiliki kompleksitas O(k × n × m), di mana k = jumlah keyword, n = panjang teks, m = panjang rata-rata keyword.

2.7. Ekspresi Regular

Ekspresi regular atau *regular expression* merupakan notasi untuk mengatakan himpunan karakter pada suatu string. Ketika sebuah *string* dapat didefinisikan dalam suatu *regular expression*, dapat dinyatakan bahwa *regular expression* tersebut cocok

dengan *string*. Sintaks yang umum digunakan dalam suatu *regular expression* merupakan standar yang digunakan pada fungsi *grep* di sistem operasi Unix atau Unix-like.

Repetitions of a pattern

*	at least 0 times	**abc*** "ab" followed by zero or more "c"
+	at least 1 time	**abc+** "ab" followed by one or more "c"
?	at most 1 time	**abc?** "ab" followed by zero or one "c"
{n}	exactly n times	**abc{2}** "ab" followed by 2 "c"
{n,}	at least n times	**abc{3,}** "ab" followed by at least 3 "c"
{n, m}	at least n times, at most m times	**abc{3,5}** "ab" followed by 3 up to 5 "c"

Positions of a pattern

^	the start of the string	**^starts** string starting with "starts"
$	the end of the string	**ends$** string ending with "ends"
\b	empty string at edge of a word	**\\bab** matches "ab" in "abc" but not in "dabc"
\B	empty string not at edge of a word	**\\Bab** matches "ab" in "dabc" but not in "abc"

Character classes I

\d	digits	**3\\d{10}** 11-digit strings starting with "3"
\D	non-digits	**a\\D** matches "ab" but not "a1"
\w	word characters (incl. underscore)	**ab\\w** matches "abc" & "ab_" but not "ab."
\W	non-word characters	**ab\\W** matches "ab." but not "abc"
\s	space	**ab\\sc** matches "ab c" but not "abc"
\S	non-space	**a\\Sc** matches "abc" but not "a c"

Character classes II

[[:digit:]] or [0-9]	digits	**ab[[:digit:]]+** matches "ab3" & "ab221"
[[:lower:]] or [a-z]	lowercase letters	**ab[[:lower:]]** matches "abc" but not "abC"
[[:upper:]] or [A-Z]	uppercase letters	**ab[[:upper:]]** matches "abC" but not "abc"
[[:alpha:]] or [A-z]	alphabetic characters	**ab[[:alpha:]]** matches "abc" & "abC"
[[:alnum:]]	alphanumeric characters	**ab[[:alnum:]]** matches "ab3", "abc" & "abC"
[[:xdigit:]]	hexadecimal characters (base 16)	**ab[[:xdigit:]]** matches "abf" but not "abg"
[[:blank:]]	blank characters (space & tab only)	**ab[[:blank:]]c** matches "ab c" & "ab\tc" but not "abc"
[[:space:]]	space characters*	**ab[[:space:]]c** matches "ab c", "ab\tc" & "ab\nc"
[[:punct:]]	punctuation characters	**ab[[:punct:]]** matches "ab!" and "ab@"
[[:graph:]]	graphical characters	**ab[[:graph:]]+** matches "ab!", "abc!" & "ab3@"
[[:print:]]	printable characters	**ab[[:print:]]+** matches "ab 3", "ab c!" & "abc"

*Space characters are tab (\t), newline (\n), vertical tab (\v), carriage return (\r), form feed (\f), and space

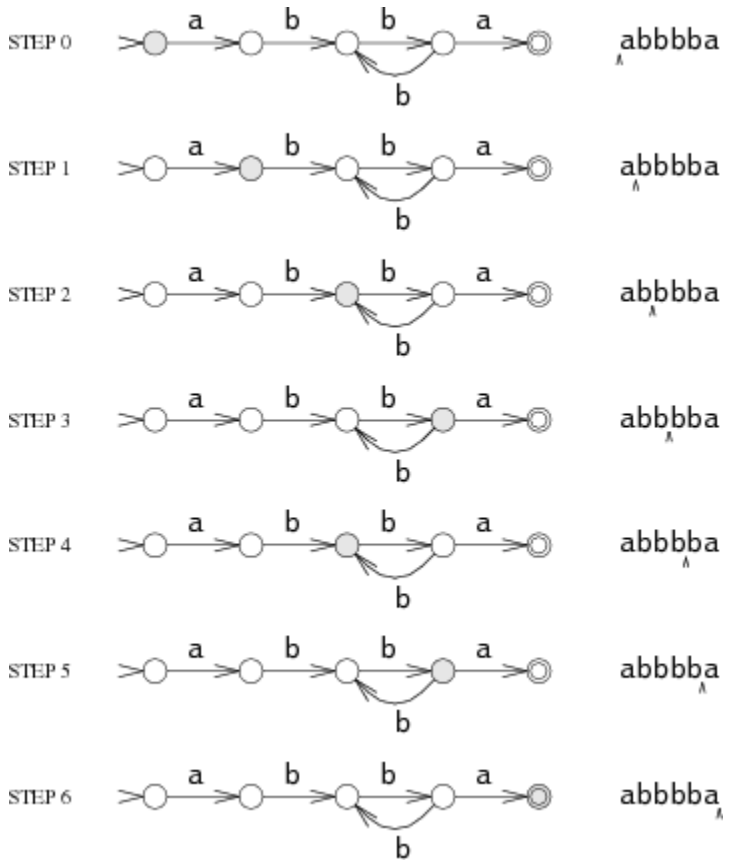
Others

.	any single character	**abc.** "abc" followed by any single character
[...]	character list	**ab[0-9]+** matches "ab3" & "ab221"
[^...]	inverted character list	**ab[^0-9]+** matches "abcd", but not "ab3d"
\	escape character	**ab*** "ab" followed by "*"
|	"or" operator	**a(b|c)** "a" followed by "b" or "c"
(...)	grouping	**a(bcd)** "a" followed by sequence "bcd"
\\N	backreference to Nth capture group	**(co)\\1** matches "coconut" but not "count"

Gambar 1.6.1. Sintaks *Regular Expression* (sumber: https://hypebright.nl/en/r-en/ultimate-cheatsheet-for-regex-in-r-2/).

Proses pencarian menggunakan *regular expression* menggunakan *finite automaton* di belakang layar. Dalam hal ini, dapat dimodelkan dalam non*deterministic finite automaton* (NFA) ataupun e-NFA. Namun, proses ini tidak akan dijelaskan dalam

dasar teori ini dan dianggap sebagai proses yang trivial. Berikut contoh pembangunan *finite automaton* untuk string “abbbba”.



Gambar 1.6.2. Contoh Pembangunan NFA pada String “abbbba” (sumber: https://swtch.com/~rsc/regexp/regexp1.html).

2.8. Browser Extension

Browser extension adalah sebuah modul perangkat lunak kecil yang menambahkan fungsionalitas khusus atau memperluas kemampuan browser web dengan memungkinkan modifikasi pada tampilan, nuansa, atau perilaku situs web. Browser extension yang kami bangun menerapkan arsitektur **Manifest V3** pada Google Chrome. Arsitektur extension ini terbagi menjadi 3 komponen utama yang modular dan saling berkomunikasi secara asinkron:

- **Content Script**

  Content script adalah skrip yang di-inject pada DOM halaman web yang sedang dibuka oleh pengguna. Komponen ini digunakan untuk

membaca dan mengekstrak teks dari node DOM, menjalankan algoritma pencarian, menerapkan highlight pada keyword yang terdeteksi, menampilkan tooltip pada keyword yang sedang di-hover, dan mendeteksi perubahan DOM secara dinamis menggunakan **MutationObserver**.

- **Background Script/Service Worker**

  Background Script adalah skrip yang berperan untuk membaca file eksternal **keyword.txt** yang berisikan keyword judol untuk pencarian algoritma di belakang layar. Komponen ini menginisialisasi pembacaan file tersebut sekali saja pada saat extension pertama kali digunakan dan menyimpan datanya ke dalam memori extension agar tidak membebani memori pada Chrome setiap kali tab baru dibuka.

- **Popup**

  Popup adalah antarmuka yang digunakan untuk menampilkan data statistik hasil deteksi keyword judol oleh setiap algoritma pencarian. Data statistik yang ditampilkan adalah berupa total keyword unik yang ditemukan, waktu eksekusi tiap algoritma, jumlah match tiap algoritma, dan perbandingan jumlah kemunculan keyword yang satu dengan lainnya. Adapun terdapat toggle untuk fitur **blurring** serta pengaktifan algoritma **Aho-Corasick** dan **Rabin-Karp**. Terakhir, terdapat tombol **rescan page** untuk mengulangi proses pencarian keyword pada halaman web yang sedang dibuka.

BAB III
ANALISIS PEMECAHAN MASALAH

3.1. Langkah-Langkah Pemecahan Masalah

Untuk membangun *chrome-extension* yang dapat melakukan *string matching* dan OCR secara paralel dengan berjalannya *browser*, terdapat sejumlah tahapan yang harus diselesaikan secara sistematis:

1. Analisis Kebutuhan Program

   Kebutuhan program secara fungsional dan nonfungsional mencakup:

   - Ekstensi *browser* yang kompatibel dengan *browser* Chromium
   - Implementasi algoritma *string matching* pada ekstensi *browser*
   - Implementasi *bypassing* CORS Middleware untuk OCR
   - Implementasi *parallel processing* pada ekstensi *browser* agar *browser* tidak *freeze* saat menjalankan algoritma dan OCR
   - Tampilan data dari ekstensi *browser* yang menunjukkan jumlah kata, kecepatan komputasi, dan kata yang *matched*
   - Visualisasi kata yang *matched*

2. Analisis Arsitektur Sistem

   Sistem ini menggunakan *layered architecture* yang terdiri dari:

   - **Presentation Layer:** Antarmuka pengguna dengan konten pada *browser* dalam bentuk ekstensi *browser*.
   - **Business Logic Layer:** Implementasi algoritma *string matching* dan OCR

3. Analisis Algoritma

   Algoritma *string matching* yang diimplementasikan meliputi:

   - Knuth-Morris-Pratt (KMP) untuk *exact matching*
   - Boyer-Moore (BM) untuk *exact matching*
   - Aho-Corasick (AC) untuk *exact matching*
   - Levenshtein Distance untuk *fuzzy matching*

3.2. Proses Pemetaan Masalah Menjadi Elemen-Elemen Algoritma Pattern Matching

3.2.1 Pemetaan ke Algoritma KMP

Permasalahan deteksi keyword judol pada halaman web dipetakan ke algoritma KMP dengan menjadikan setiap keyword dari keyword.txt sebagai pattern dan keseluruhan teks hasil ekstraksi DOM (flatTextMap) sebagai teks. Elemen-elemen algoritma KMP dipetakan sebagai berikut. Border function dan failure function digunakan untuk memproses setiap keyword sebelum pencarian dimulai. Untuk setiap keyword, dihitung nilai LPS (Longest Proper Prefix which is also Suffix) yang merepresentasikan seberapa jauh pointer pattern dapat digeser saat terjadi mismatch, tanpa harus mengulang perbandingan dari awal. Misalnya keyword SCATTER menghasilkan LPS array [0,0,0,0,0,0,0] karena tidak ada prefix yang sekaligus menjadi suffix, sehingga setiap mismatch menggeser pointer sepenuhnya. Nilai LPS ini dihitung sekali per keyword dan digunakan sepanjang proses pencarian di seluruh flatTextMap.

Proses pencarian menggunakan dua pointer, pointer i pada teks dan pointer j pada pattern, yang bergerak maju secara bersamaan saat karakter cocok. Saat terjadi mismatch dengan j > 0, pointer j tidak direset ke 0 melainkan digeser ke lps[j-1], memanfaatkan informasi prefix-suffix yang sudah dihitung. Setiap kali j mencapai panjang pattern, sebuah match ditemukan dan posisinya dicatat sebagai SearchResult dengan startIndex = i - j dan endIndex = i.

3.2.2 Pemetaan ke Algoritma Boyer-Moore

Serupa dengan KMP, Boyer-Moore menjadikan setiap keyword sebagai pattern dan flatTextMap sebagai teks, namun melakukan perbandingan dari kanan ke kiri pada pattern. Last occurrence table dibangun dari setiap keyword sebelum pencarian dimulai. Tabel ini memetakan setiap karakter yang muncul dalam keyword ke indeks kemunculan terakhirnya. Misalnya untuk keyword GACOR, tabel

menghasilkan {G:0, A:1, C:2, O:3, R:4}. Karakter yang tidak muncul dalam keyword diperlakukan sebagai -1.

Shifting process menggunakan nilai dari last occurrence table untuk menentukan seberapa jauh pattern digeser saat terjadi mismatch. Jika karakter c pada teks yang menyebabkan mismatch ditemukan di dalam pattern, pattern digeser sehingga kemunculan terakhir c di pattern sejajar dengan posisi mismatch di teks. Jika c tidak ada di pattern, pattern digeser sepenuhnya melewati posisi tersebut. Besar pergeseran dihitung sebagai max(1, j - last[c]) untuk memastikan pattern selalu bergerak maju.

3.2.3 Pemetaan ke Algoritma Aho-Corasick

Berbeda dengan KMP dan BM yang memproses satu keyword per eksekusi, Aho-Corasick memetakan seluruh keyword sekaligus sebagai pattern melalui struktur automaton. Pembangunan trie dilakukan dengan memasukkan semua keyword ke dalam pohon prefiks. Setiap node merepresentasikan satu karakter, dan setiap path dari root ke leaf merepresentasikan satu keyword. Keyword judol yang memiliki prefiks yang sama, misalnya jika ada GACOR dan GACORBET, akan berbagi node pada bagian prefiks yang identik, menghemat ruang dan waktu preprocessing.

Failure links dipetakan dari konsep failure function KMP ke struktur trie. Untuk setiap node u, failure link menunjuk ke node yang merepresentasikan suffix terpanjang dari string di u yang juga merupakan prefiks dari keyword lain. Nilai ini dihitung via BFS satu kali saat fase inisialisasi. Saat pencarian berlangsung dan karakter teks tidak cocok dengan edge yang tersedia, pointer bergerak mengikuti failure link — persis seperti pointer j pada KMP yang bergerak ke lps[j-1].

Output links memungkinkan sistem melaporkan semua keyword yang cocok di setiap posisi teks secara efisien, termasuk keyword yang merupakan suffix dari keyword lain. Saat pointer berada di suatu node,

sistem menelusuri seluruh output link chain untuk mengumpulkan semua keyword yang berakhir di posisi tersebut.

3.2.4 Pemetaan ke Weighted Levenshtein Distance

Fuzzy matching dipetakan ke permasalahan deteksi dengan menjadikan setiap token dari flatTextMap sebagai s1 dan setiap keyword sebagai s2 dalam perhitungan jarak edit. Tokenisasi menggunakan regex /[a-zA-Z0-9\u0080-\uFFFF]+/g yang memecah flatTextMap pada tanda baca dan whitespace sambil mempertahankan karakter unicode. Setiap token menyimpan startIndex dan endIndex yang mengacu ke posisi aslinya di flatTextMap, sehingga hasil fuzzy matching dapat langsung digunakan untuk highlight di DOM.

Tabel substitusi berbobot memetakan realita manipulasi visual pada konten judol dimana pengguna yang mengetik SL0T bermaksud menyamarkan SLOT. Pasangan karakter seperti (O, 0), (A, 4), (I, 1), serta homoglyph unicode seperti (M, M) dan (S, S) diberi bobot substitusi 0.2, sehingga jarak edit antara SL0T dan SLOT hanya 0.2 alih-alih 1.0. Ini membuat similarity score tetap tinggi meskipun ada substitusi visual.

Threshold similarity 0.75 memetakan batas keputusan antara "token ini adalah keyword yang disamarkan" dan "token ini adalah kata biasa yang kebetulan mirip". Nilai ini dipilih berdasarkan analisis bahwa kata empat karakter dengan satu substitusi normal menghasilkan similarity ≈ 0.75, sedangkan kata dengan satu substitusi visual menghasilkan similarity ≈ 0.95, keduanya di atas threshold, sementara kata yang benar-benar berbeda menghasilkan similarity di bawah 0.5.

3.2.5 Pemetaan ke Algoritma Rabin-Karp

Rabin-Karp dipetakan ke permasalahan deteksi dengan menggunakan nilai hash sebagai representasi setiap keyword dan setiap window teks berukuran sama. Polynomial rolling hash memetakan setiap keyword menjadi sebuah bilangan bulat menggunakan fungsi hash hash(s)

= Σ s[i] · B^(m-1-i) mod P dengan basis B = 31 dan modulus prima P = 1.000.000.007. Saat window pencarian bergeser satu karakter di flatTextMap, nilai hash diperbarui dalam O(1) dengan mengurangi kontribusi karakter yang keluar dan menambahkan kontribusi karakter yang masuk tanpa menghitung ulang seluruh window.

Verifikasi karakter dipetakan sebagai mekanisme pengaman terhadap spurious hit, kondisi di mana dua string berbeda menghasilkan nilai hash yang sama akibat collision. Setiap kali hash window cocok dengan hash keyword, sistem melakukan verifikasi karakter per karakter untuk memastikan match yang sesungguhnya sebelum mencatat hasilnya sebagai SearchResult.

3.3. Fitur Fungsional dan Arsitektur Extension yang Dibangun

Extension yang kami bangun menerapkan arsitektur pipeline linear agar manipulasi DOM tidak merusak layout asli halaman web. Adapun extension memiliki beberapa fitur fungsional yaitu sebagai berikut:

- Menerapkan **TreeWalker** untuk mengekstrak semua node teks pada elemen DOM halaman web lalu menggabungkannya dalam sebuah string besar dan dipetakan dengan indeks. Terkadang, sebuah halaman web memecah satu kata ke dalam beberapa tag HTML bersarang sehingga kami menerapkan ini.
- Menerapkan **MutationObserver** untuk mendukung pencarian keyword pada halaman web yang bisa saja dapat di-scrolling tanpa henti misalnya pada platform sosial media seperti X/instagram. Extension akan memindai node baru yang terdapat pada layar tanpa memindai ulang seluruh halaman web.
- Menggunakan **chrome.tabs.sendMessage** untuk state management dan komunikasi real-time pada fitur **blur** dan **rescan** agar tidak perlu refresh halaman web untuk eksekusi fitur tersebut.
- Memunculkan tooltip berisikan detail hasil deteksi saat melakukan hover pada keyword yang di-highlight pada halaman web. Extension kami

menerapkan pendekatan **DOM Traversal Aggregation** dimana saat pengguna melakukan hover pada satu kata yang di-highlight, extension melakukan query ke elemen parent (misalnya atau) lalu menjumlahkan total waktu eksekusi dan jumlah keyword yang ditemukan tiap algoritma yang berada dalam blok teks yang sama. Hal ini memastikan informasi statistik merepresentasikan agregasi performa secara akurat per level TextNode.

3.4. Contoh Ilustrasi Kasus

- Algoritma:

  Situs judi seringkali menyamarkan kata kunci agar lolos dari sensor sistem, misalnya menulis kata "gacor" menjadi "G4c0r" (mengganti 'a' dengan '4', 'o' dengan '0') atau "Gαc0r" (menggunakan huruf Yunani Alpha 'α'). Jika hanya bergantung pada algoritma pencocokan string eksak (seperti KMP atau Boyer-Moore), kata penyamaran ini tidak akan terdeteksi sama sekali karena karakter penyusunnya secara biner berbeda.

  Solusi yang kami implementasikan adalah menerapkan algoritma Fuzzy Search dengan Weighted Levenshtein Distance. Algoritma ini memotong kalimat menjadi token kata dan menghitung jarak editnya. Kami menetapkan matriks bobot khusus (visual similarity pairs), di mana biaya operasi penggantian (substitution cost) karakter yang mirip secara visual (seperti a → 4 atau a → α) diturunkan dari bobot standar 1.0 menjadi hanya 0.1. Dengan demikian, jarak edit antara "gacor" dan "Gαc0r" bernilai sangat kecil (0.2) sehingga menghasilkan skor kemiripan yang sangat tinggi (di atas ambang batas 0.85). Hasilnya, sistem dapat mendeteksi kata penyamaran tersebut secara akurat dan menyensornya.

- Extension: Kasus elemen HTML bersarang

  Misalnya terdapat kata “slot” yang dimasukkan pada halaman web dengan format ** slot **. Jika hanya bergantung pada algoritma pencarian saja, kata ini tidak akan terdeteksi karena terpisah oleh tag HTML Solusi yang

kami implementasikan pada kasus ini adalah menerapkan **TreeWalker** pada extension yang kami bangun. TreeWalker ini akan mengekstrak keempat huruf tersebut yang merupakan node teks lalu menggabungkannya menjadi satu teks “slot” dan dipetakan menggunakan indeks. Hasil pemetaannya adalah huruf s pada indeks 0, huruf l pada indeks 1, dan ot pada indeks 2. Jika sebuah algoritma pencarian menemukan kata “slot” pada indeks 0-3, hasil pemetaan tersebut akan digunakan untuk membungkus masing-masing pecahan node menjadi satu keyword untuk di-highlight tanpa perlu menghilangkan tag <b> di tengahnya.

BAB IV
IMPLEMENTASI DAN PENGUJIAN

4.1. *Pseudocode* Algoritma

4.1.1. Knuth-Morris-Pratt

Algorithm 1 Knuth-Morris-Pratt (KMP) Algorithm

```
1:  procedure ComputeLPS(P)                          ▷ P: Pattern of length m
2:      lps ← array of size m filled with 0
3:      len ← 0
4:      i ← 1
5:      while i < m do
6:          if P[i] = P[len] then
7:              len ← len + 1
8:              lps[i] ← len
9:              i ← i + 1
10:         else
11:             if len ≠ 0 then
12:                 len ← lps[len − 1]
13:             else
14:                 lps[i] ← 0
15:                 i ← i + 1
16:             end if
17:         end if
18:     end while
19:     return lps
20: end procedure

21: procedure KMPSearch(T, P)        ▷ T: Text of length n, P: Pattern of
    length m
22:     lps ← ComputeLPS(P)
23:     i ← 0                                        ▷ Index for Text
24:     j ← 0                                        ▷ Index for Pattern
25:     results ← Empty List
26:     while i < n do
27:         if T[i] = P[j] then
28:             i ← i + 1
29:             j ← j + 1
30:         end if
31:         if j = m then
32:             AddMatch(results, i − j, i)
33:             j ← lps[j − 1] i < n and T[i] ≠ P[j]
34:             if j ≠ 0 then
35:                 j ← lps[j − 1]
36:             else
37:                 i ← i + 1
38:             end if
39:         end if
40:     end while
41:     return results
42: end procedure
```

Input: Teks Utama (T), Pola Pencarian (P).

Output: Daftar indeks awal lokasi kecocokan pola P di dalam teks T.

Langkah-langkah Proses:

1. Pra-pemrosesan (Tabel LPS): Sistem menghitung panjang pola (m) dan teks (n), lalu membuat tabel Longest Prefix Suffix (LPS) berukuran m. Tabel ini mencatat panjang prefix pola terpanjang yang juga merupakan suffix dari sub-pola tersebut.
2. Tetapkan penunjuk posisi teks (i=0) dan penunjuk posisi pola (j=0).
3. Karakter T[i] dibandingkan dengan P[j] dalam perulangan hingga akhir teks (i<n):
    - Jika karakter cocok (T[i]==P[j]), kedua penunjuk digeser ke kanan (i=i+1,j=j+1).
    - Jika seluruh pola berhasil dicocokkan (j==m), posisi indeks awal kecocokan dicatat pada posisi i−j. Penunjuk pola kemudian digeser ke j=LPS[j−1] untuk mencari kecocokan berikutnya tanpa menghentikan pemindaian.
4. Jika terjadi ketidakcocokan (T[i]=P[j]):
    - Apabila j = 0, penunjuk pola digeser mundur ke j=LPS[j−1] (menghindari pemeriksaan ulang dari awal pola), sedangkan penunjuk teks i tetap diam.
    - Apabila j == 0, penunjuk teks bergeser ke kanan (i=i+1).

4.1.2. Boyer-Moore

Algorithm 2 Boyer-Moore (Bad Character) Algorithm

```
1: procedure BuildBadCharTable(P)                ▷ P: Pattern of length m
2:     badChar ← Map of character to integer
3:     for i ← 0 to m − 1 do
4:         badChar[P[i]] ← i
5:     end for
6:     return badChar
7: end procedure

8: procedure BoyerMooreSearch(T, P)              ▷ T: Text of length n, P:
   Pattern of length m
9:     badChar ← BuildBadCharTable(P)
10:    results ← Empty List
11:    s ← 0                                     ▷ s: pattern shift relative to text
12:    while s ≤ n − m do
13:        j ← m − 1
14:        while j ≥ 0 and P[j] = T[s + j] do
15:            j ← j − 1
16:        end while
17:        if j < 0 then
18:            AddMatch(results, s, s + m)
19:            if s + m < n then
20:                s ← s + m − badChar.get(T[s + m], −1)
21:            else
22:                s ← s + 1
23:            end if
24:        else
25:            badCharShift ← j − badChar.get(T[s + j], −1)
26:            s ← s + Max(1, badCharShift)
27:        end if
28:    end while
29:    return results
30: end procedure
```

Input: Teks Utama (T), Pola Pencarian (P).

Output: Daftar indeks awal kecocokan pola P pada teks T.

Langkah-langkah Proses:

1. Dibuat tabel pemetaan karakter (Bad Character Table) dari pola P. Setiap karakter unik dalam pola dipetakan ke indeks kemunculan terakhirnya. Karakter yang tidak ada dalam pola diberi nilai default -1.
2. Tetapkan posisi pergeseran pola relatif terhadap teks (s=0).

3. Selama jendela pola masih berada di dalam batas panjang teks ($s \leq n-m$):
    - Set indeks pembanding $j=m-1$ (memulai pencocokan dari karakter paling kanan pola).
    - Bandingkan karakter secara mundur ke kiri (P[j] dengan T[s+j]) selama karakter cocok dan indeks $j \geq 0$.
4. Keputusan Pergeseran:
    - Jika pola cocok sepenuhnya ($j<0$), posisi pergeseran saat ini (s) dicatat sebagai kecocokan. Pola kemudian digeser berdasarkan heuristik karakter setelah jendela pola.
    - Jika terjadi mismatch (P[j] =T[s+j]), hitung nilai pergeseran, shift=j−BadCharTable[T[s+j]]. Pola digeser ke kanan sejauh max(1,shift) untuk perbandingan berikutnya.

4.1.3. Regular Expression

```
const regex = /\b([\p{L}]+)(\d{2,3})\b/giu;
```

Algorithm 3 Regular Expression Matching

```
1:  procedure RegexSearch(T, R)                ▷ T: Text, R: Regex Pattern
2:      results ← Empty List
3:      regex ← CompileRegex(R, Global Flag)
4:      match ← ExecRegex(regex, T)
5:      while match ≠ Null do
6:          start ← match.index
7:          end ← start + Length(match[0])
8:          AddMatch(results, match[0], start, end)
9:          match ← ExecRegex(regex, T)
10:     end while
11:     return results
12: end procedure
```

Input: Teks Utama (T), Pola Ekspresi Reguler (R).

Output: Daftar substring yang cocok beserta indeks awal dan akhirnya.

Langkah-langkah Proses:

1. Kompilator regex menerjemahkan pola ekspresi reguler R ke dalam bentuk Finite State Automaton (baik berupa NFA maupun DFA).
2. Mesin regex membaca teks utama T karakter demi karakter dari kiri ke kanan untuk memicu transisi status pada automata sesuai dengan aturan simbol kelas karakter, grup penangkap, atau kuantifier.
3. Jika mesin menemui jalan buntu (mismatch status sebelum pola regex terpenuhi secara penuh), mesin akan mundur (backtrack) ke status percabangan sebelumnya untuk mencoba rute transisi alternatif.
4. Ketika status akhir (match) tercapai, mesin mencatat substring teks yang cocok, indeks posisi awal, dan indeks posisi akhirnya. Pemindaian dilanjutkan dari titik akhir kecocokan tersebut jika flag global diaktifkan.

4.1.4. Fuzzy Matching

Algorithm 4 Weighted Levenshtein and Fuzzy Search

```
procedure SubstitutionCost(a, b)
    if a = b then
        return 0.0
    end if
    if (a, b) is visually similar then
        return 0.1
    end if
    return 1.0
end procedure

procedure WeightedLevenshtein(S1, S2) ▷ S1, S2: Strings to compare
    n1 ← Length(S1), n2 ← Length(S2)
    dp ← 2D Array of size (n1 + 1) × (n2 + 1)
    for i ← 0 to n1 do
        dp[i][0] ← i × 1.0
    end for
    for j ← 0 to n2 do
        dp[0][j] ← j × 1.0
    end for
    for i ← 1 to n1 do
        for j ← 1 to n2 do
            cost ← SubstitutionCost(S1[i − 1], S2[j − 1])
            dp[i][j] ← Min(dp[i − 1][j] + 1.0, dp[i][j − 1] + 1.0, dp[i − 1][j − 1] +
cost)
        end for
    end for
    return dp[n1][n2]
end procedure

procedure FuzzySearch(T, K, θ)  ▷ T: Text, K: Keywords, θ: similarity
threshold
    tokens ← Tokenize(T)
    results ← Empty List
    for each token in tokens do
        for each keyword in K do
            distance ← WeightedLevenshtein(token.text, keyword)
            maxLen ← Max(token.length, keyword.length)
            score ← 1.0 − (distance/maxLen)
            if score ≥ θ then
                AddMatch(results, token.start, token.end)
            end if
        end for
    end for
    return results
end procedure
```

Input: Kata Dokumen CV (W), Kata Kunci Kamus (K).

Output: Skor kemiripan (similarity score) antara kata dokumen dan kata kunci kamus.

Langkah-langkah Proses:

1. Teks kandidat dipotong menjadi daftar kata individual (token).
2. Membuat matriks berukuran $(|W|+1)\times(|K|+1)$ untuk menyimpan akumulasi biaya perubahan. Baris dan kolom ke-0 diisi secara berurutan sebagai representasi biaya operasi penyisipan (insertion) dan penghapusan (deletion).
3. Dilakukan iterasi di setiap sel (i,j) untuk membandingkan karakter W[i−1] dengan K[j−1]:
    - Jika karakter sama, biaya penggantian (substitution cost) adalah 0.0.
    - Jika karakter berbeda tetapi mirip secara visual (seperti o dengan 0, atau a dengan α), biaya penggantian diturunkan menjadi 0.1 (toleransi leetspeak).
    - Jika karakter berbeda secara total, biaya penggantian adalah 1.0.
    - Sel matriks dp[i][j] diisi dengan nilai minimum dari hasil operasi: atas (hapus), kiri (sisip), atau diagonal (ganti) ditambah biaya masing-masing.
4. Nilai pojok kanan bawah matriks merupakan jarak akhir (distance). Skor kemiripan dihitung dengan rumus: $1.0 - (distance/max(|W|,|K|))$. Jika skor memenuhi ambang batas, dinyatakan cocok.

4.1.5. Aho-Corasick

Algorithm 5 Aho-Corasick Automaton Search

```
1:  procedure BuildAutomaton(K)                          ▷ K: List of Keywords
2:      root ← CreateNode
3:      for each keyword in K do
4:          curr ← root
5:          for each ch in keyword do
6:              if ch ∉ curr.children then
7:                  curr.children[ch] ← CreateNode
8:              end if
9:              curr ← curr.children[ch]
10:         end for
11:         curr.outputs.Add(keyword)
12:     end for
13:     queue ← Empty Queue
14:     for each child in root.children do
15:         child.failLink ← root
16:         queue.Enqueue(child)
17:     end for
18:     while queue is not empty do
19:         curr ← queue.Dequeue
20:         for each ch, child in curr.children do
21:             fail ← curr.failLink
22:             while fail ≠ root and ch ∉ fail.children do
23:                 fail ← fail.failLink
24:             end while
25:             child.failLink ← fail.children[ch] or root
26:             child.outputLinks ← child.failLink.outputs is not empty ? child.failLink :
    child.failLink.outputLinks
27:             queue.Enqueue(child)
28:         end for
29:     end while
30:     return root
31: end procedure

32: procedure AhoCorasickSearch(T, K)        ▷ T: Text, K: Keywords list
33:     root ← BuildAutomaton(K)
34:     curr ← root
35:     results ← Empty List
36:     for i ← 0 to Length(T) − 1 do
37:         ch ← T[i]
38:         while curr ≠ root and ch ∉ curr.children do
39:             curr ← curr.failLink
40:         end while
41:         if ch ∈ curr.children then
42:             curr ← curr.children[ch]
43:         end if
44:         out ← curr
45:         while out ≠ Null do
46:             for each keyword in out.outputs do
47:                 start ← i − keyword.length + 1
48:                 AddMatch(results, start, i + 1)
49:             end for
50:             out ← out.outputLinks
51:         end while
52:     end for
```

Input: Teks Utama (T), Daftar Kata Kunci Kamus (K).

Output: Daftar seluruh kata kunci kamus yang ditemukan beserta posisinya pada teks.

Langkah-langkah Proses:

1. Semua kata kunci dari daftar K dimasukkan ke dalam struktur data Trie. Simpul terminal yang menandai akhir kata kunci diberi label daftar output kata tersebut.
2. Dilakukan penelusuran BFS pada Trie untuk menghubungkan setiap simpul dengan Failure Link (tautan kegagalan ke simpul terpanjang yang merupakan suffix-nya) dan Output Link (tautan jalan pintas ke simpul lain yang menghasilkan kecocokan kata).
3. Ditentukan penunjuk status pada simpul root Trie. Sistem menyusuri teks utama T karakter demi karakter:
    - Jika karakter saat ini memiliki transisi status di simpul saat ini, ikuti transisi ke simpul anak.
    - Jika tidak ada transisi, ikuti tautan Failure Link berulang kali hingga menemukan transisi yang valid atau kembali ke root.
4. Pada setiap langkah perpindahan simpul, periksa daftar output simpul dan susuri rantai Output Link untuk mencatat seluruh kata kunci yang berhasil dicocokkan di posisi indeks teks saat ini secara sekaligus.

4.1.6. Rabin-Karp

Algorithm 6 Rabin-Karp Rolling Hash Algorithm

```
1:  procedure RabinKarpSearch(T, P, d, q)        ▷ T: Text, P: Pattern, d: Radix, q: Prime Modulo
2:      n ← Length(T)
3:      m ← Length(P)
4:      h ← d^(m−1) (mod q)
5:      p ← 0                                    ▷ Hash for pattern
6:      t ← 0                                    ▷ Hash for text window
7:      results ← Empty List
8:      for i ← 0 to m − 1 do
9:          p ← (d × p + P[i]) (mod q)
10:         t ← (d × t + T[i]) (mod q)
11:     end for
12:     for s ← 0 to n − m do
13:         if p = t then
14:             if T[s . . . s + m − 1] = P then
15:                 AddMatch(results, s, s + m)
16:             end if
17:         end if
18:         if s < n − m then
19:             t ← (d × (t − T[s] × h) + T[s + m]) (mod q)
20:             if t < 0 then
21:                 t ← t + q
22:             end if
23:         end if
24:     end for
25:     return results
26: end procedure
```

Input: Teks Utama (T), Pola Kata Kunci (P), Konstanta Basis (d), Konstanta Modulo (q).

Output: Indeks awal kemunculan pola pada teks.

Langkah-langkah Proses:

1. Hitung nilai hash awal untuk pola P dan jendela teks awal T[0…m−1] menggunakan skema polinomial modulo q.
2. Bandingkan nilai hash pola dengan nilai hash jendela teks berjalan:
   - Jika nilai hash sama, lakukan verifikasi karakter demi karakter langsung secara manual antara P dan jendela teks untuk mengantisipasi terjadinya bentrokan nilai hash (collision). Jika cocok, catat indeks saat ini.
3. Ketika jendela digeser satu karakter ke kanan, hitung nilai hash jendela berikutnya secara efisien dalam waktu O(1) dengan membuang komponen nilai karakter paling kiri yang keluar dan mengalikan basis untuk menambahkan nilai karakter baru yang masuk:
   $hash_{baru} = (d \times (hash_{lama} - T[kiri] \times d^{m-1}) + T[baru]) (mod\ q)$

Jika nilai hash bernilai negatif, tambahkan konstanta modulo q untuk memulihkannya ke bilangan positif. Lanjutkan perulangan hingga akhir dokumen.

4.2. Spesifikasi Teknis Program

Bagian ini menjelaskan mengenai struktur proyek, struktur data, fungsi, dan prosedur yang bangun untuk implementasi program kami.

4.2.1. Struktur Proyek

```
Tubes3_kakeguruistimashou
├── dist/
├── doc/
│   └── kakeguruistimashou.pdf
├── keywords/
│   └── keywords.txt
├── public/
│   ├── tesseract/
│   │   ├── eng.traineddata.gz
│   │   ├── tesseract-core.wasm
│   │   ├── tesseract-core.wasm.js
│   │   └── worker.min.js
│   ├── manifest.json
│   └── offscreen.html
├── src/
│   ├── algorithms/
│   │   ├── algorithm.ts
│   │   ├── bonus.ts
│   │   └── fuzzy.ts
│   ├── background/
│   │   ├── background.ts
│   │   └── offscreen.ts
│   ├── content/
│   │   ├── content.ts
│   │   ├── dom_walker.ts
│   │   └── highlight.ts
│   ├── ocr/
│   │   └── ocr.ts
│   ├── popup/
│   │   ├── popup.html
│   │   └── popup.ts
│   ├── styles/
│   │   ├── content.css
│   │   └── popup.css
│   ├── interface.ts
│   └── vite-env.d.ts
```

```
├── test/
│   ├── fuzzy-normal.html
│   └── testcases.html
├── .gitignore
├── package-lock.json
├── package.json
├── README.md
├── tsconfig.json
└── vite.config.ts
```

Berikut adalah penjelasan dari masing-masing direktori/folder dan file:

- **dist/**: Folder hasil kompilasi/build yang siap di-load di Chrome
- **doc/**: Folder berisikan laporan pengerjaan proyek
- **keywords.txt**: File berisikan keyword yang digunakan pada algoritma pencarian
- **public/**: Folder yang berisikan aset-aset statis yang akan disalin langsung ke folder dist oleh Vite tanpa melalui proses kompilasi atau modifikasi
    - **tesseract/**: Folder yang berisikan pustaka utama (library) Tesseract.js secara lokal (termasuk worker.min.js, tesseract-core.wasm.js, tesseract-core.wasm, dan data bahasa eng.traineddata.gz) untuk menjalankan fitur OCR secara offline tanpa dependensi eksternal
    - **manifest.json:** File manifest ekstensi Chrome (Manifest V3) yang mendefinisikan metadata ekstensi, hak akses izin (permissions), file background script, content script, dan konfigurasi popup ekstensi
    - **offscreen.html:** Halaman HTML minimal yang digunakan sebagai wadah Offscreen Document agar Tesseract.js WebAssembly dapat dikompilasi dan dijalankan secara aman di latar belakang tanpa melanggar aturan Content Security Policy (CSP) halaman web eksternal
- **src/**: Folder berisikan kode utama program
    - **interface.ts:** File berisikan struktur data yang digunakan pada extension

- **algorithm.ts**: File yang berisikan kelas pencocokan string Matcher yang mengimplementasikan pencarian eksak menggunakan algoritma KMP (Knuth-Morris-Pratt), Boyer-Moore, dan pencarian pola menggunakan Regular Expression (RegEx)
- **bonus.ts**: File yang berisikan implementasi algoritma string matching bonus, yaitu Aho-Corasick (pencocokan multi-pola secara efisien menggunakan trie automaton) dan Rabin-Karp (pencocokan pola berbasis hashing dengan rolling hash)
- **fuzzy.ts**: File yang berisikan logika pencarian kabur (fuzzy search) dengan toleransi kemiripan visual karakter (untuk mendeteksi bypass leetspeak/unicode) menggunakan metode kalkulasi jarak kata Weighted Levenshtein Distance
- **background.ts**: Background service worker ekstensi yang bertugas untuk memuat kamus kata dari keywords.txt ke penyimpanan lokal chrome, mendirikan Offscreen Document, serta menjembatani komunikasi pesan OCR antara content script dan offscreen script
- **offscreen.ts**: Script yang berjalan di dalam Offscreen Document yang bertugas menerima perintah jalankan OCR dari background script, memanggil proses OCR lokal, dan mengirimkan hasil ekstraksi teks kembali
- **content.ts**: File berisikan logika utama pencarian keyword pada halaman web termasuk highlight/blur keyword, tooltip, OCR, dan MutationObserver untuk menangani scrolling
- **dom_walker.ts**: File berisikan fungsi untuk mengekstrak node teks dari elemen HTML menggunakan TreeWalker
- **highlight.ts**: File berisikan kelas untuk menerapkan highlight pada keyword yang terdeteksi oleh algoritma pencarian
- **ocr.ts**: File penolong (helper) yang mendefinisikan inisialisasi worker offline Tesseract.js, proses pengambilan gambar sebagai

binary Blob (untuk melewati blokir CORS & bug base64 decoder), dan ekstraksi teks/kalimat dari gambar
    - **popup.html**: Struktur HTML popup extension berisikan data statistik dan visualisasi hasil pencarian keyword pada halaman web
    - **popup.ts**: File berisikan logika fitur-fitur pada popup
    - **content.css**: Design dari hasil highlight dan blur keyword serta tampilan tooltip
    - **popup.css**: Design dari popup extension
- **test/**: Folder berisikan template HTML untuk pengujian dan solusi jawaban pengujian program
    - **fuzzy-normal.html**: Halaman HTML sederhana yang berisikan variasi teks leetspeak dan simbol unicode seperti G4COR, sl0t, Gαcor999 untuk menguji efektivitas deteksi dari algoritma fuzzy search
    - **testcases.html**: Kumpulan kasus uji (test cases) lengkap dalam bentuk HTML terstruktur untuk melakukan verifikasi hasil deteksi KMP, BM, RegEx, Levenshtein, dan false-positive secara langsung di browser
- **.gitignore:** File berisikan komponen yang tidak diikutsertakan pada repositori program
- **package.json & package-lock.json:** File berisikan *library/dependency* yang digunakan pada program
- **README.md:** Dokumentasi proyek berisikan penjelasan singkat algoritma KMP dan BM yang diimplementasikan, *requirement* program dan instalasi, langkah-langkah *build* dan cara load extension di Chrome, serta identitas pembuat
- **tsconfig.json:** File konfigurasi utama TypeScript yang menentukan aturan kompilasi kode seperti target output ESNext, resolusi modul NodeNext, mode type-checking ketat, serta registrasi tipe API Chrome

- **vite.config.ts:** File konfigurasi bundler Vite untuk mendefinisikan target entri build ekstensi (popup, content, background, offscreen) dan memuat plugin pembersih folder output (chromeExtensionPlugin) guna membetulkan tautan path stylesheet dan script pada file final.

4.2.2. Struktur Data

Berikut adalah beberapa struktur data yang digunakan pada program ini:

```
// Struktur data hasil pencarian algoritma
export interface SearchResult {
    keyword: string;
    algorithm: string;
    startIndex: number;
    endIndex: number;
    occurrences: number;
    executionTime: number;
    timeDetails?: string; // detail waktu eksekusi tiap
algoritma
    occurrenceDetails?: string; // detail jumlah kemunculan
tiap keyword
    occurrenceMap?: Record<string, Record<string, number>>;
// map kemunculan keyword per algoritma
}
```

Interface SearchResult adalah struktur data yang digunakan untuk menyimpan dan mengagregasi data dari setiap penemuan kata kunci judol. Atribut yang dimiliki:

- keyword: Kata kunci yang terdeteksi
- algorithm: Algoritma yang menemukan kata kunci
- startIndex & endIndex: Indeks posisi awal dan akhir kata kunci dalam peta teks gabungan yang digunakan untuk menentukan elemen DOM yang akan di-highlight
- occurences: Total jumlah kemunculan keyword
- executionTime: Waktu eksekusi yang diperlukan oleh algoritma

- timeDetails, occurenceDetails, & occurenceMap: Atribut opsional yang digunakan untuk menyimpan hasil yang telah diformat dan diagregasi untuk ditampilkan secara detail pada tooltip dan statistik popup

```
// struktur data utk mapping karakter dengan node teks DOM
export interface TextNodeMap {
  node: Text;
  startIndex: number;
  endIndex: number;
}
```

Interface TextNodeMap adalah struktur data yang digunakan untuk memetakan karakter teks yang diekstrak dari halaman web kembali ke Text Node DOM asalnya. Struktur data ini berperan penting dalam proses highlight karena memungkinkan extension untuk membungkus bagian teks yang cocok dengan kata kunci judol menggunakan elemen tanpa merusak struktur HTML yang bersarang. Atribut yang dimiliki:

- node: Objek Text dari DOM tempat teks berada
- startIndex & endIndex: Indeks awal dan akhir karakter teks pada node ini dalam peta teks gabungan

Pemetaan ini memungkinkan algoritma pencarian yang bekerja pada satu string besar untuk mengetahui secara tepat Text Node mana dan pada posisi offset berapa highlight harus diterapkan.

```
// struktur data statistik yang disimpan di storage
export interface StorageData {
  judolStats?: {
    lastScanTime: number;
    totalKeywords: number;
    totalExecutionTime: number;
    detailedResults: SearchResult[];
  };
}
```

Interface StorageData adalah struktur data yang digunakan untuk menyimpan data statistik hasil deteksi dalam penyimpanan lokal/storage ekstensi. Interface ini memiliki atribut opsional judolStats yang mengagregasi semua data yang diperlukan untuk ditampilkan di popup ekstensi:

- lastScanTime: Waktu pemindaian terakhir dilakukan
- totalKeywords: Total jumlah kata kunci unik yang terdeteksi di keseluruhan halaman
- totalExecutionTime: Total waktu eksekusi yang diperlukan oleh semua algoritma pencarian
- detailedResults: Array yang berisi objek

```
// struktur data utk visualisasi perbandingan hasil pencarian
algoritma
export interface ChartData {
  label: string;
  value: number;
  color: string;
}
```

Interface ChartData adalah struktur data yang digunakan untuk visualisasi perbandingan hasil pencarian tiap algoritma. Struktur ini menampung tiga atribut utama yang diperlukan untuk membuat elemen grafik yang ditampilkan di popup ekstensi:

- label: String yang berfungsi sebagai label untuk nama algoritma pencarian
- value: Number yang merepresentasikan nilai data, misalnya total jumlah kata kunci yang ditemukan oleh algoritma tersebut
- color: String warna visualisasi untuk data poin tersebut dalam grafik

4.2.3. Fungsi dan Prosedur

- content.ts

```ts
/**
 * Menghitung dan menetapkan posisi koordinat tooltip secara dinamis di layar agar
 * tooltip deteksi tidak meluap keluar dari batas pandang layar browser (viewport).
 *
 * @param event Objek event pergerakan mouse (MouseEvent) saat melakukan hover pada kata kunci.
 */
function positionTooltip(event: MouseEvent): void {
    // Menghitung koordinat X dan Y berdasarkan kursor mouse.
    // Melakukan penyesuaian posisi jika tooltip melebihi batas window.innerWidth atau window.innerHeight.
    pass
}

/**
 * Menyimpan data hasil temuan kata kunci (baik teks maupun gambar) ke dalam Chrome Storage lokal
 * untuk kemudian dibaca dan divisualisasikan oleh panel statistik popup ekstensi.
 *
 * @param newResults Array hasil pencarian baru yang ditemukan.
 * @param isInitialScan Penanda apakah pemindaian ini merupakan pemindaian awal halaman (clean scan).
 */
async function saveResultsToStorage(newResults: SearchResult[], isInitialScan: boolean = false): Promise<void> {
    // Membaca data statistik yang telah ada di storage.
    // Menggabungkan hasil temuan baru dan memperbarui total waktu eksekusi global.
    // Menyimpan kembali data terperbarui ke local storage.
    pass
}

/**
 * Menyaring daftar hasil temuan kata kunci pada indeks posisi awal yang sama agar hanya mengambil
 * hasil pencocokan dengan string terpanjang (menghindari substring pendek terdeteksi ganda).
 *
 * @param results Daftar hasil pencarian mentah.
 * @returns Daftar hasil pencarian yang sudah difilter dari substring pendek.
 */
function filterSwallowedMatches(results: SearchResult[]): SearchResult[] {
    // Mengelompokkan hasil pencarian berdasarkan startIndex yang sama.
    // Mengurutkan setiap kelompok dan hanya mengambil elemen dengan endIndex terpanjang.
    pass
}
```

```ts
/**
 * Menyaring hasil temuan pola agar tidak ada duplikasi atau tumpang tindih indeks posisi
 * antar hasil deteksi yang berbeda pada area teks yang sama.
 *
 * @param results Daftar hasil pencarian.
 * @returns Daftar hasil pencarian bersih tanpa adanya tumpang tindih indeks posisi.
 */
export function filterOverlappingMatches(results: SearchResult[]): SearchResult[] {
    // Mengurutkan temuan berdasarkan startIndex dan menyeleksi elemen yang tidak saling bertabrakan posisi indeksny
    pass
}

/**
 * Mengelompokkan dan menggabungkan hasil temuan yang saling bertumpang tindih (overlap)
 * ke dalam satu representasi klaster pencarian tunggal yang mencatat semua algoritma dan kata kunci terkait.
 *
 * @param results Daftar hasil pencarian mentah dari semua algoritma.
 * @returns Daftar klaster hasil pencarian gabungan untuk ditampilkan pada tooltip.
 */
export function mergeOverlappingMatches(results: SearchResult[]): SearchResult[] {
    // Melakukan pengelompokan temuan yang berdekatan atau bersinggungan ke dalam klaster (cluster).
    // Menghitung rincian waktu, frekuensi kemunculan lokal, dan peta algoritma untuk tooltip.
    pass
}

/**
 * Memulai proses OCR asinkron pada elemen gambar, menyaring hasil menggunakan
 * string matching, dan mengaburkan gambar jika ditemukan kata kunci judi.
 *
 * @param img Elemen HTMLImageElement target gambar yang akan dipindai.
 * @param keywordsToSearch Daftar kata kunci kamus sasaran pencarian.
 */
async function processImageOCR(img: HTMLImageElement, keywordsToSearch: string[]): Promise<void> {
    // Mengirim pesan PERFORM_OCR ke background script dan menunggu hasil ekstraksi teks.
    // Memproses kalimat hasil ekstraksi gambar menggunakan KMP, BM, RegEx, dan Fuzzy.
    // Menerapkan class sensor (blur) dan menyimpan hasilnya ke storage jika terbukti mencocokkan kata judi.
    pass
}
```

```ts
/**
 * Fungsi utama halaman konten ekstensi untuk memindai seluruh elemen teks DOM dan gambar di bawah rootElement,
 * mengevaluasi kecocokan dengan seluruh algoritma pencarian, serta menerapkan penyorotan/sensor.
 *
 * @param rootElement Node elemen DOM HTML target pemindaian.
 * @param isInitialScan Penanda apakah scan dilakukan pertama kali saat halaman dimuat.
 */
async function scanPage(rootElement: HTMLElement = document.body, isInitialScan: boolean = false): Promise<void> {
    // Menjalankan OCR pada gambar di bawah rootElement secara paralel.
    // Mengekstrak teks halaman menggunakan buildTextMap.
    // Menjalankan pencarian teks menggunakan RegEx, KMP, Boyer-Moore, Aho-Corasick, Rabin-Karp, dan Fuzzy.
    // Menggabungkan temuan tumpang tindih dan menerapkan visualisasi highlight dan blur.
    pass
}

/**
 * Fungsi utilitas untuk menetapkan status sensor (blur) default menjadi aktif (true) saat ekstensi
 * pertama kali digunakan oleh browser pengguna.
 */
async function enableDefaultBlur(): Promise<void> {
    // Memeriksa status isBlurEnabled pada storage.
    // Jika belum didefinisikan, tetapkan nilai awal default bernilai true.
    pass
}
```

- background.ts

```ts
/**
 * Fungsi untuk memuat daftar kata kunci mentah dari keywords.txt ke dalam penyimpanan local ekstensi (caching).
 * Dijalankan otomatis ketika ekstensi diinstal atau browser baru saja dinyalakan.
 */
async function initializeDictionary(): Promise<void> {
    // Membaca kata kunci mentah dari file dan menyimpannya ke local storage menggunakan chrome.storage.local.
    pass
}

/**
 * Melakukan parsing pada string teks mentah dari file keywords.txt menjadi array berisi daftar kata kunci bersih.
 *
 * @param rawText Isi teks mentah dari file keywords.txt.
 * @returns Array berisi kata kunci tanpa spasi tambahan di awal/akhir dan mengabaikan baris kosong.
 */
function parseTextFile(rawText: string): string[] {
    // Memecah baris teks menggunakan regex baris baru.
    // Memangkas spasi tambahan dan menyaring baris-baris kosong.
    pass
}

/**
 * Memastikan Offscreen Document untuk pemrosesan OCR sudah aktif. Jika belum ada,
 * fungsi akan membuat dokumen baru secara asinkron menggunakan modul offscreen.
 */
async function ensureOffscreenDocument(): Promise<void> {
    // Memeriksa keberadaan dokumen offscreen yang aktif melalui tab chrome.
    // Jika belum ada, memanggil chrome.offscreen.createDocument dengan tipe worker OCR.
    pass
}
```

- dom_walker.ts

```ts
/**
 * Fungsi untuk mengambil dan memetakan semua node teks (TextNode) dari elemen root HTML
 * yang diberikan menggunakan TreeWalker. Mengabaikan tag script, style, noscript, serta string kosong.
 *
 * @param rootElement Elemen HTML teratas tempat pemindaian dimulai.
 * @returns Objek berisi gabungan teks mentah (flatTextMap) dan peta simpul teks (textNodeMap).
 */
export function buildTextMap(rootElement: HTMLElement): { flatTextMap: string; textNodeMap: TextNodeMap[] } {
    // Membuat TreeWalker untuk menelusuri simpul teks yang memenuhi filter.
    // Menghitung indeks posisi awal dan akhir dari setiap simpul teks relatif terhadap teks gabungan.
    // Mengembalikan gabungan teks dan peta simpul teks.
    pass
}
```

- highlight.ts

```ts
/**
 * Kelas utilitas statis yang bertanggung jawab untuk menerapkan dan menghapus
 * penyorotan (highlight) dan efek sensor (blur) kata kunci judi pada DOM halaman web.
 */
export class Highlighter {
    ...

    /**
     * Menerapkan efek penyorotan (highlight) dan sensor (blur) pada semua simpul teks
     * di halaman web yang beririsan dengan rentang posisi indeks kata kunci judi yang ditemukan.
     *
     * @param match Objek data hasil temuan kata kunci (SearchResult) yang berisi rentang indeks dan informasi algoritma.
     * @param shadowMap Peta koordinat indeks posisi dari seluruh simpul teks (TextNode) yang telah dipindai oleh dom_walker.
     * @param blurEnabled Status apakah efek sensor blur diaktifkan pada halaman web (default: false).
     */
    public static applyHighlight(match: SearchResult, shadowMap: TextNodeMap[], blurEnabled = false): void {
        // Menyaring simpul teks dari shadowMap yang indeks awal/akhirnya beririsan dengan indeks temuan match.
        // Menghitung koordinat posisi relatif indeks kata kunci terhadap masing-masing simpul teks yang beririsan.
        // Memanggil metode wrapNode untuk menyisipkan elemen penanda (highlight).
        pass
    }
```

```
    private static wrapNode(node: Text, start: number, end: number, blurEnabled: boolean, match: SearchResult): void {
        // Membagi simpul teks menjadi tiga bagian (sebelum, kata kunci, sesudah) menggunakan splitText.
        // Membuat elemen span baru dengan kelas CSS '.judol-highlight'.
        // Jika blurEnabled aktif, tambahkan kelas '.is-blurred' pada elemen span.
        // Menyimpan data statistik kata kunci (keyword, algoritma, occurrences, time) ke dalam dataset span.
        // Menggantikan simpul teks kata kunci asli dengan elemen span tersebut di dalam DOM.
        pass
    }

    /**
     * Menghapus seluruh penyorotan kata kunci (highlight) dan sensor (blur) dari halaman web,·
     * serta mengembalikan susunan simpul teks dokumen DOM ke keadaan semula (bersih).
     */
    public static removeAllHighlights(): void {
        // Mencari semua elemen span dengan kelas '.judol-highlight' di dalam dokumen body.
        // Melepas elemen pembungkus span dengan memindahkan simpul teks anak kembali ke elemen induknya.
        // Menghapus elemen span kosong tersebut dari dokumen.
        // Menjalankan document.body.normalize() untuk menyatukan kembali simpul teks yang terpecah akibat proses splitText.
        pass
    }
```

- ocr.ts

```
/**
 * Fungsi utama untuk melakukan proses optical character recognition (OCR) secara offline·
 * pada gambar menggunakan Tesseract.js worker yang berjalan di context offscreen document.
 *·
 * @param imageSrc Tautan URL atau data URI base64 dari gambar yang akan diproses.
 * @returns Janji (Promise) berisi teks hasil deteksi dari gambar.
 */
export async function detectTextFromImage(imageSrc: string): Promise<string> {
    // Mengunduh gambar sebagai Blob untuk menghindari blokir CORS.
    // Menginisialisasi Tesseract Worker offline menggunakan aset-aset lokal ekstensi.
    // Menjalankan proses pengenalan karakter dan mengembalikan hasil ekstraksi teks.
    pass
}

/**
 * Memecah teks besar hasil OCR gambar menjadi baris-baris kalimat individual untuk diproses lebih lanjut.
 *·
 * @param text Teks hasil ekstraksi gambar secara keseluruhan.
 * @returns Array kalimat-kalimat terpisah yang tidak kosong.
 */
export function detectSentences(text: string): string[] {
    // Membagi teks menggunakan ekspresi reguler pemisah kalimat (. ! ? \n \r).
    // Menghilangkan spasi berlebih pada awal/akhir kalimat dan membuang string kosong.
    pass
}
```

- popup.ts

```
/**
 * Menggambar bagan visualisasi batang (Bar Chart) perbandingan frekuensi kemunculan
 * kata kunci yang ditemukan oleh masing-masing algoritma pada canvas HTML5.
 *
 * @param statsArray Array berisi data perbandingan jumlah temuan masing-masing algoritma.
 */
function drawComparisonChart(statsArray: ChartData[]): void {
    // Mendapatkan context 2D dari elemen canvas HTML.
    // Menyesuaikan rasio piksel layar (devicePixelRatio) agar rendering chart tajam.
    // Menggambar batang berwarna representatif dan menuliskan label nama algoritma serta jumlahnya.
    pass
}

/**
 * Membaca data statistik terbaru dari Chrome Storage dan memperbarui tampilan antarmuka (UI) popup
 * (seperti jumlah kata kunci unik, rincian waktu eksekusi, grafik batang, dan daftar kemunculan kata kunci).
 *
 * @param stats Struktur data statistik judolStats yang tersimpan di storage.
 */
export async function renderStatistics(stats?: StorageData['judolStats']): Promise<void> {
    // Membaca preferensi algoritma bonus yang aktif dari storage.
    // Menyaring hasil pencarian hanya dari algoritma yang diaktifkan oleh pengguna.
    // Menghitung jumlah keyword unik, waktu eksekusi rata-rata, dan memperbarui visualisasi chart dan daftar keywor
    pass
}
```

4.3. Tata Cara Penggunaan Extension

1. Kompilasi extension

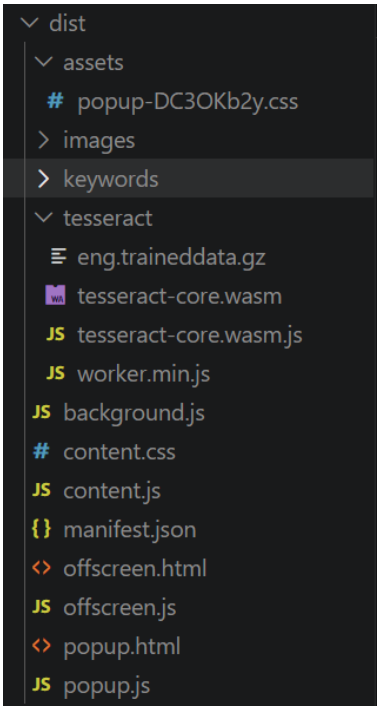
Sebelumnya, perlu dilakukan cloning repositori github pada sebuah folder lokal dengan memasukkan command berikut:

```
git clone https://github.com/aufafaza/Tubes3_kakeguruistimashou.git
```

Buka sebuah terminal pada folder tempat clone repositori tersebut dan masukkan command berikut untuk kompilasi extension:

```
npx vite build
```

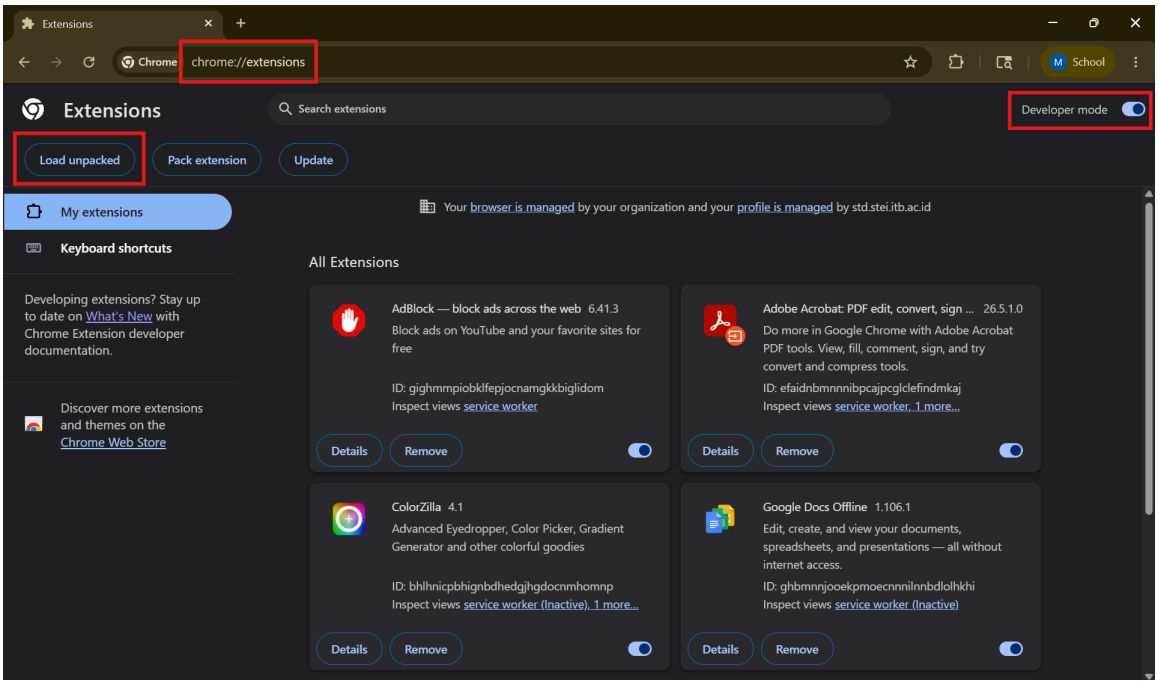
Kompilasi akan menghasilkan sebuah folder **dist** yang kurang lebih isinya seperti pada gambar berikut:



Gambar 4.2.1 Isi folder dist

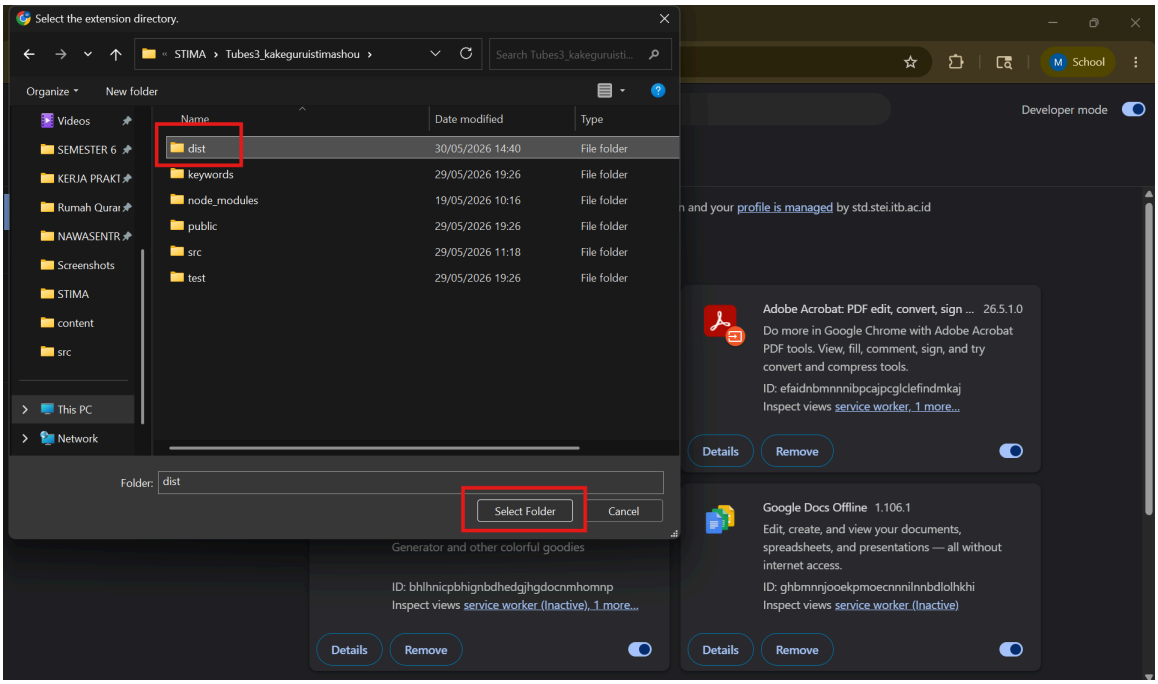
2. **Upload Extension**

   Buka Google Chrome dan masukkan URL **chrome://extensions/** untuk masuk ke bagian extension pada browser. Kemudian, aktifkan **Developer mode** dengan klik toggle pada pada pojok kanan atas.



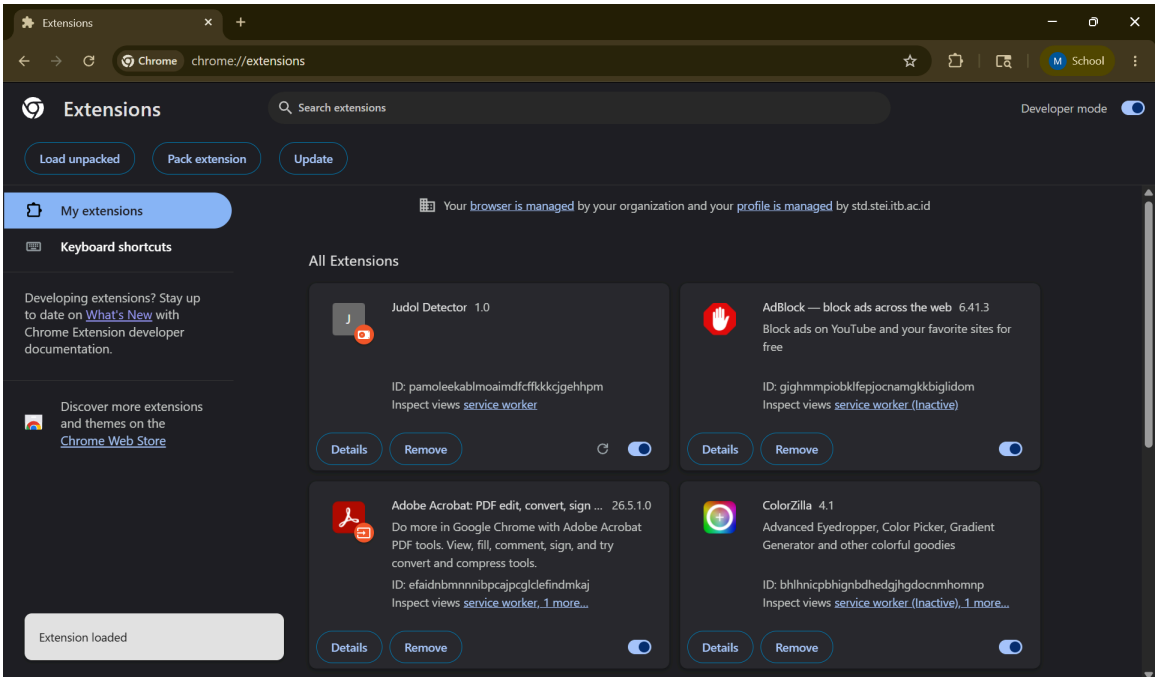
Gambar 4.2.2 Cara upload extension ke browser Google Chrome (1)

Lalu, klik tombol **Load unpacked** untuk upload folder **dist** dari hasil kompilasi sebelumnya.



Gambar 4.2 Cara upload extension ke browser Google Chrome (2)

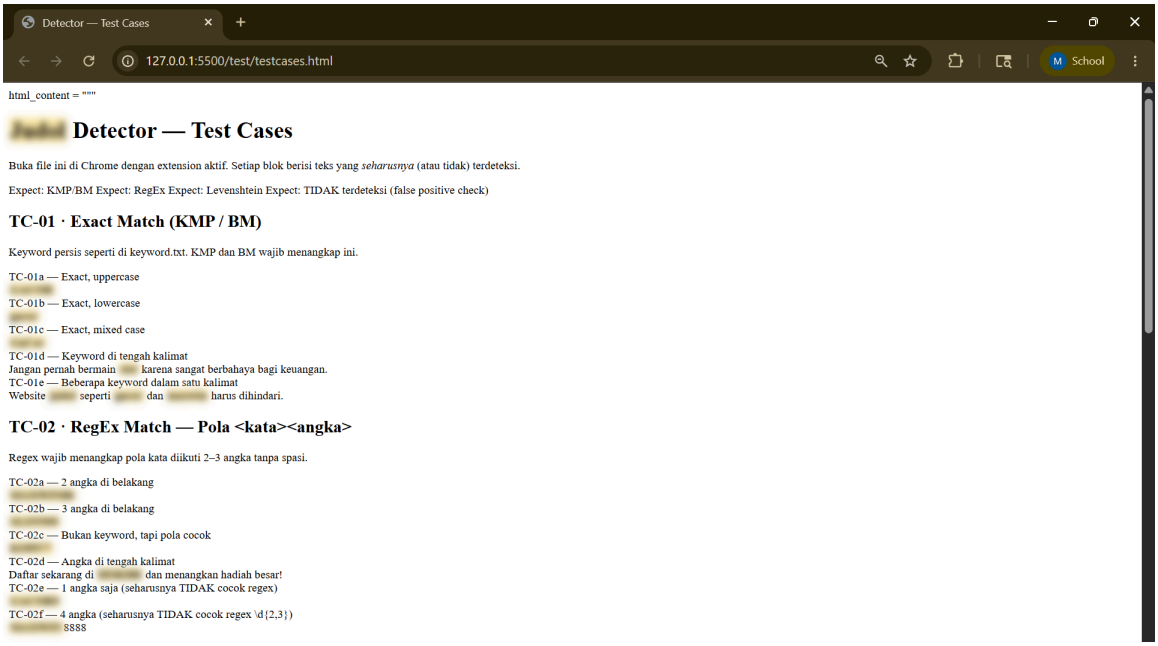
Setelah extension berhasil diupload, tampilannya akan seperti pada gambar berikut:



Gambar 4.2.3 Upload extension ke browser Google Chrome berhasil

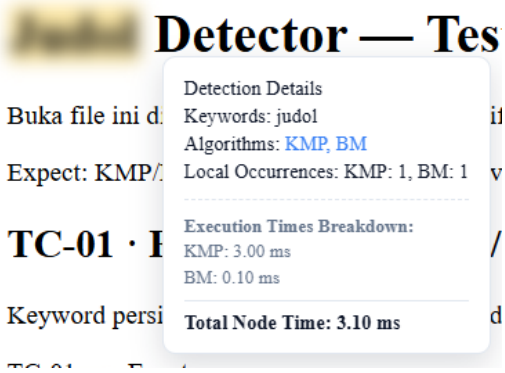
3. Pemindaian Keyword

Extension akan berjalan otomatis saat pengguna membuka halaman web baru dan keyword yang terdeteksi akan langsung di-highlight dan di-blur.



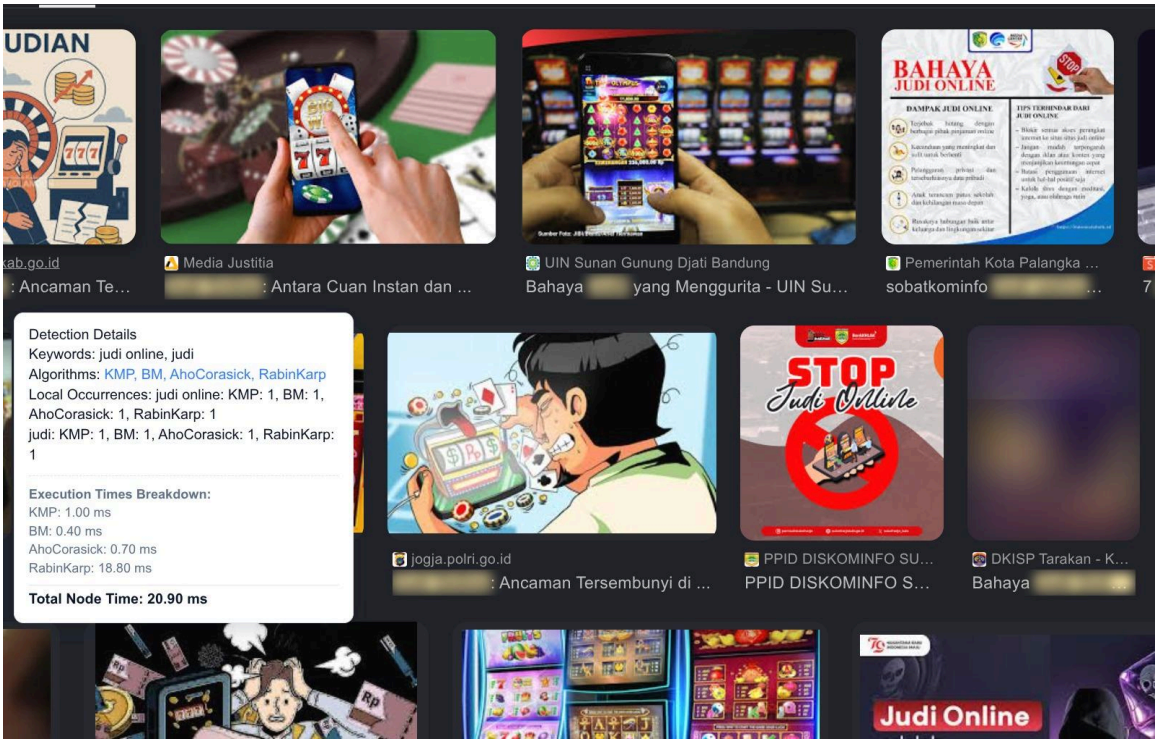
Gambar 4.2.4 Contoh hasil pemindaian keyword oleh algoritma pencarian pada sebuah halaman web

Jika pengguna mengarahkan kursor (hover) ke kata yang di-highlight, akan muncul **Tooltip** yang data detail hasil deteksi keyword oleh algoritma pencarian. Data yang ditampilkan adalah keyword yang terdeteksi, algoritma yang menemukan, total kemunculan pada node teks tersebut, dan total waktu eksekusi algoritma.



Gambar 4.2.5 Tampilan tooltip

Jika terdapat gambar pada halaman web yang mengandung keyword judol, fitur OCR akan melakukan blur pada gambar tersebut.



Gambar 4.2.6 Contoh hasil blur gambar oleh fitur OCR

4. Penggunaan Popup

Untuk membuka popup, pengguna dapat icon extension Judol Detector pada *toolbar* Chrome. Berikut adalah contoh tampilan popup yang menampilkan data statistik hasil deteksi keyword:



Gambar 4.2.7 Tampilan popup

Berikut adalah beberapa fitur yang disediakan pada popup:

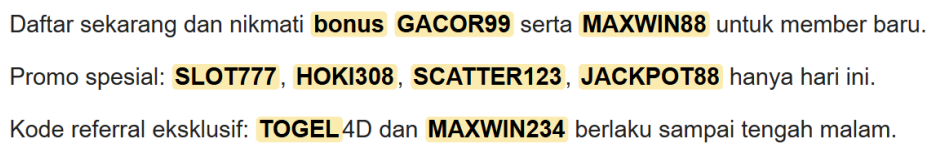
- **Total keyword unik** yang terdeteksi di keseluruhan halaman web saat ini
- Total **waktu eksekusi tiap algoritma** pencarian
- Grafik **perbandingan jumlah keyword** yang terdeteksi oleh **tiap algoritma** pencarian
- Visualisasi j**umlah kemunculan tiap keyword**
- Toggle untuk mengaktifkan/menonaktifkan fitur **blur** keyword yang terdeteksi serta pencarian algoritma **Aho-Corasick** dan **Rabin Karp**
- Tombol **Rescan Page** untuk mengulangi proses pemindaian halaman dari awal

4.4. Hasil dan Analisis Pengujian

4.4.1. Exact Matching dengan KMP / BM / Aho-Corasick

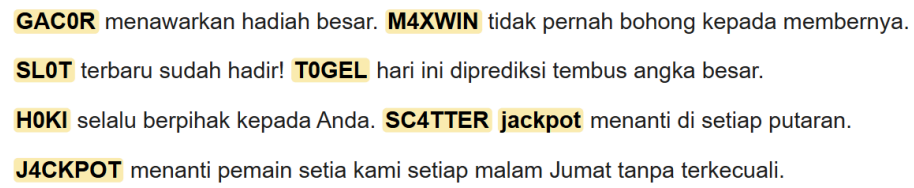
Selamat datang di situs **GACOR** terpercaya. Kami menyediakan layanan **SLOT** dan **TOGEL** resmi. Dapatkan **JACKPOT** besar bersama kami. Promo **MAXWIN** dan **SCATTER** berlaku hari ini! Bergabunglah dengan ribuan member **HOKI** yang sudah merasakan manfaatnya.

4.4.2. Regex + Rabin-Karp (pola <kata><angka>)



Bagian ini menguji kemampuan sistem mendeteksi keyword yang digabungkan dengan angka, seperti GACOR99, SLOT777, dan MAXWIN234. Pola ini umum digunakan di situs judol sebagai variasi nama platform atau kode promo. Deteksi dilakukan oleh dua mekanisme secara bersamaan. Regex dengan pola \b([\p{L}]+)(\d{2,3})\b menangkap pola kata diikuti 2–3 digit angka sebagai satu kesatuan token. Rabin-Karp melengkapi deteksi ini dengan pendekatan berbasis rolling hash untuk setiap keyword yang terdaftar. Seluruh tujuh pola berhasil terdeteksi, termasuk MAXWIN234 dengan tiga digit dan TOGEL4D meskipun suffix angkanya hanya satu digit setelah huruf, yang berarti regex menangkap TOGEL sebagai exact match lebih dulu.

4.4.3. Fuzzy Matching Substitusi Visual



Bagian ini menguji ketahanan sistem terhadap manipulasi karakter yang umum dilakukan untuk menghindari deteksi, yaitu mengganti huruf dengan angka atau simbol yang mirip secara visual: 0→O, 4→A, 1→I. Weighted Levenshtein Distance menghitung jarak edit antara token dan keyword, dengan bobot substitusi visual sebesar 0.2, jauh lebih kecil dari substitusi normal (1.0). Hal ini menyebabkan similarity score token seperti SL0T terhadap SLOT mencapai ≈ 0.96, jauh di atas threshold 0.75. Seluruh delapan varian manipulasi berhasil terdeteksi, termasuk GAC0R setelah ditambahkan ke keyword.txt sebagai solusi pragmatis untuk kasus di mana exact match substring menghalangi fuzzy matching.

4.4.4. Keyword di Dalam Tabel

Platform	Bonus	Keterangan
GACOR 99	MAXWIN 88	Promo aktif hari ini
SL0T 777	JACKPOT	Fuzzy + exact match
TOGEL 4D	SCATTER 123	Regex pattern match
HOKI 308	M4XWIN	Visual substitution

Pengujian ini memverifikasi bahwa sistem dapat mendeteksi keyword yang berada di dalam elemen <table>, <tr>, dan <td>. Seluruh keyword dalam tabel berhasil ter-highlight, membuktikan bahwa TreeWalker di buildTextMap berhasil menelusuri struktur DOM tabel yang lebih kompleks.

4.4.5. Fuzzy Matching Unicode Homoglyph

GACOR hadir dengan tampilan baru yang lebih menarik dan modern untuk semua kalangan.

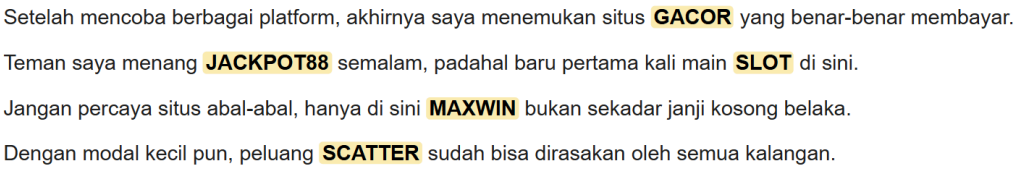
HOKI memberikan pengalaman bermain yang tak terlupakan bagi seluruh membernya.

SLOT online terbaik se-Asia kini hadir di genggaman tangan Anda setiap saat.

MAXWIN dijamin oleh sistem keamanan berlapis yang tidak bisa ditembus siapapun.

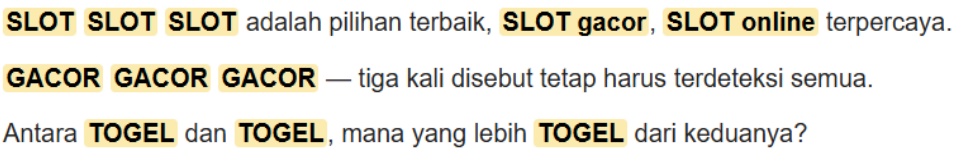
Bagian ini menguji skenario yang lebih canggih, yaitu penggunaan karakter dari alfabet lain yang bentuknya identik atau sangat mirip dengan huruf Latin, seperti Alpha Yunani (A), Eta Yunani (H), Cyrillic Dze (S), dan Mu Yunani (M). Sistem berhasil mendeteksi keempat kata (GACOR, HOKI, SLOT, MAXWIN) karena pasangan homoglyph tersebut terdaftar di VISUAL_SIMILAR_PAIRS dengan bobot substitusi 0.2. Tokenizer berbasis regex /[a-zA-Z0-9\u0080-\uFFFF]+/g juga berperan penting di sini dengan range \u0080-\uFFFF memastikan karakter unicode non-ASCII seperti A, H, S, dan M tetap dianggap bagian dari token, bukan dibuang sebagai karakter asing.

4.4.6. Konteks Kalimat Realistis



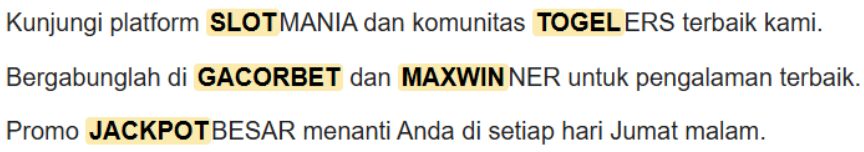
Bagian ini mensimulasikan kondisi nyata di mana keyword judol muncul di tengah kalimat panjang yang wajar secara linguistik, bukan terisolasi. Semua keyword berhasil terdeteksi meskipun dikelilingi kata-kata biasa. Ini memvalidasi bahwa buildTextMap dan sistem tokenisasi bekerja dengan benar dan teks dari seluruh node DOM digabungkan menjadi flatTextMap, dan indeks posisi tiap karakter tetap akurat sehingga highlight di DOM tidak meleset. Kasus JACKPOT88 juga berhasil ditangkap sebagai satu kesatuan pola <kata><angka> oleh regex.

4.4.7. Edge Case: Keyword muncul berkali-kali dalam satu kalimat



Pengujian ini memverifikasi bahwa sistem melanjutkan pencarian setelah menemukan match pertama, bukan berhenti di kemunculan pertama saja. Kalimat yang mengandung kata SLOT sebanyak lima kali dan GACOR sebanyak tiga kali diuji, dan seluruh kemunculan berhasil ter-highlight secara independen. Ini membuktikan bahwa mekanisme lps[j-1] pada KMP dan kelanjutan loop pada Aho-Corasick berfungsi dengan benar.

4.4.8. Edge Case: Keyword sebagai Substring dari Kata Lain



Pengujian ini mengeksplorasi perilaku sistem ketika keyword merupakan substring dari kata yang lebih panjang, seperti SLOT dalam

SLOTMANIA atau GACOR dalam GACORBET. Sistem mendeteksi dan meng-highlight bagian keyword tersebut karena implementasi tidak menggunakan word boundary secara eksplisit pada exact matching. Perilaku ini merupakan trade-off desain yang perlu dicatat — di satu sisi meningkatkan recall dengan menangkap lebih banyak varian, di sisi lain berpotensi menghasilkan false positive pada kata-kata yang kebetulan mengandung substring keyword.

4.4.9. Edge Case: Case Sensitivity

slot gacor maxwin togel jackpot scatter hoki

Slot Gacor Maxwin Togel Jackpot Scatter Hoki

sLoT gAcOr mAxWiN tOgEl jAcKpOt sCaTtEr hOkI

SLOT GACOR MAXWIN TOGEL JACKPOT SCATTER HOKI

Pengujian ini memverifikasi konsistensi case-insensitive matching di seluruh algoritma. Empat baris yang mengandung keyword yang sama namun dengan kombinasi kapital yang berbeda — semua lowercase, title case, mixed case acak, dan semua uppercase — diuji sekaligus. Seluruh kemunculan berhasil ter-highlight secara identik, membuktikan bahwa konversi toLowerCase() diterapkan secara konsisten di KMP, Boyer-Moore, Aho-Corasick, dan Rabin-Karp.

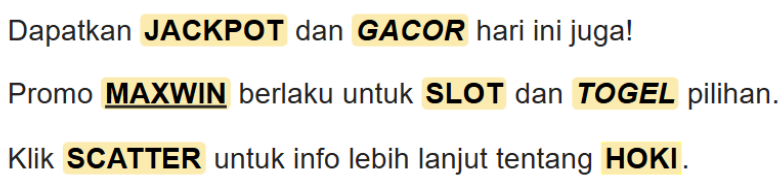
4.4.10. Edge Case: Teks Panjang dengan Keyword tersebar

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. GACOR Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum. Sed ut perspiciatis unde omnis iste natus error sit voluptatem accusantium doloremque laudantium, totam rem aperiam. SLOT Eaque ipsa quae ab illo inventore veritatis et quasi architecto beatae vitae dicta sunt explicabo. Nemo enim ipsam voluptatem quia voluptas sit aspernatur aut odit aut fugit, sed quia consequuntur magni dolores eos qui ratione. TOGEL Neque porro quisquam est, qui dolorem ipsum quia dolor sit amet, consectetur, adipisci velit, sed quia non numquam. Ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim ad minima veniam, quis nostrum exercitationem ullam. MAXWIN Quis autem vel eum iure reprehenderit qui in ea voluptate velit esse quam nihil molestiae consequatur, vel illum qui. At vero eos et accusamus et iusto odio dignissimos ducimus qui blanditiis praesentium voluptatum deleniti atque. JACKPOT Nam libero tempore, cum soluta nobis est eligendi optio cumque nihil impedit quo minus id quod maxime placeat facere. Temporibus autem quibusdam et aut officiis debitis aut rerum necessitatibus saepe eveniet ut et voluptates repudiandae. SCATTER Itaque earum rerum hic tenetur a sapiente delectus, ut aut reiciendis voluptatibus maiores alias consequatur aut. HOKI

Pengujian ini mensimulasikan halaman web berkonten panjang (~300 kata) di mana tujuh keyword tersebar secara acak di antara teks Latin yang tidak relevan. Seluruh keyword berhasil terdeteksi tanpa ada

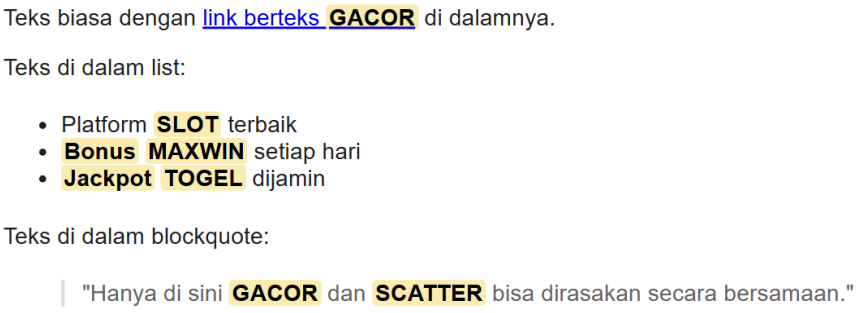
yang terlewat, memvalidasi bahwa sistem mampu bekerja dengan benar pada volume teks yang lebih besar dari sekadar kalimat pendek.

4.4.11. Edge Case: Keyword di Elemen Nested



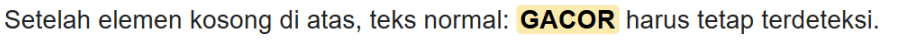
Pengujian ini memverifikasi bahwa TreeWalker di buildTextMap berhasil menelusuri seluruh child node terlepas dari tag HTML yang membungkusnya. Keyword yang ditempatkan di dalam , , <u>, <b>, <i>, , dan <mark> seluruhnya berhasil terdeteksi dan ter-highlight, membuktikan bahwa sistem tidak terbatas pada teks di level <p> saja.

4.4.12. Edge Case: Keyword di Elemen Interaktif dan Struktural



Pengujian ini memperluas pengujian pada elemen struktural sebelumnya ke elemen-elemen struktural yang lebih beragam, yaitu <a>, <li>, dan <blockquote>. Seluruh keyword di dalam elemen-elemen tersebut berhasil terdeteksi, memvalidasi bahwa jangkauan TreeWalker mencakup seluruh pohon DOM tanpa terkecuali.

4.4.13. Edge Case: Teks Kosong, Whitespace, dan Node Trivial



Pengujian ini memverifikasi ketahanan sistem terhadap node-node trivial — paragraf kosong, paragraf berisi spasi saja, dan elemen tanpa konten. Sistem tidak mengalami crash dan keyword pada baris terakhir tetap terdeteksi dengan benar, membuktikan bahwa pengecekan flatTextMap.trim() === "" dan filter whitespace di TreeWalker berfungsi sebagai guard yang efektif.

4.4.14. Edge Case: Keyword di Antara Tanda Baca

(GACOR) [SLOT] {TOGEL} "MAXWIN" 'JACKPOT'

GACOR, SLOT. TOGEL! MAXWIN? JACKPOT:

GACOR—SLOT–TOGEL...MAXWIN

GACOR/SLOT\TOGEL|MAXWIN

Pengujian ini memverifikasi presisi range highlight ketika keyword dikelilingi berbagai jenis tanda baca — tanda kurung, koma, titik, tanda seru, tanda tanya, slash, dan dash. Sistem berhasil meng-highlight hanya bagian keyword yang tepat tanpa menyertakan tanda baca di sekitarnya, membuktikan bahwa kalkulasi localStartIndex dan localEndIndex di highlight.ts akurat.

4.4.15. Edge Case: Keyword Overlapping dan Adjacent

GACORSLOT — dua keyword langsung bersebelahan tanpa spasi.

SLOTTOGEL — apakah keduanya terdeteksi?

MAXWINJACKPOT — dua keyword panjang bersebelahan.

Pengujian ini menguji perilaku filterOverlappingMatches ketika dua keyword berada persis bersebelahan tanpa pemisah. Pada kasus GACORSLOT, sistem mendeteksi GACOR di posisi awal dan SLOT di posisi setelahnya secara berurutan — keduanya berhasil ter-highlight karena range-nya tidak overlap melainkan adjacent. Hasil ini membuktikan bahwa logika filter overlap hanya membuang hasil yang benar-benar tumpang tindih, bukan yang bersebelahan.

4.4.16. Edge Case: Kombinasi Fuzzy + Tanda Baca

(SL0T) [GAC0R] {M4XWIN} "T0GEL" 'J4CKPOT'

SL0T, GAC0R. M4XWIN! T0GEL? J4CKPOT:

Pengujian ini menggabungkan tantangan dari EC-10 dan TC-03 sekaligus — varian fuzzy seperti SL0T dan GAC0R yang diapit oleh tanda baca. Tokenizer berbasis regex /[a-zA-Z0-9\u0080-\uFFFF]+/g berhasil memisahkan token dari tanda baca di sekitarnya, sehingga SL0T dalam (SL0T) tetap diekstrak sebagai token bersih dan berhasil dicocokkan oleh fuzzy matching. Seluruh varian berhasil terdeteksi, memvalidasi bahwa perbaikan tokenizer memberikan dampak positif tidak hanya untuk teks biasa tetapi juga untuk kombinasi dengan tanda baca.

BAB V
KESIMPULAN DAN SARAN

5.1. Kesimpulan

Pada Tugas Besar 3 IF2211 Strategi Algoritma ini, algoritma string matching berupa Knuth-Morris-Pratt (KMP), Boyer-Moore (BM), dan Regex diimplementasikan dalam sebuah Chrome Extension untuk melakukan pencarian kata kunci secara langsung pada halaman web yang sedang dibuka pengguna. Algoritma-algoritma ini bekerja dengan cara mencocokkan pattern yang dimasukkan pengguna terhadap teks yang diekstraksi dari konten halaman secara real-time.

Kami berhasil mengimplementasikan ketiga algoritma string matching tersebut ke dalam ekstensi berbasis browser. Selain mengimplementasikan algoritma yang diwajibkan, ekstensi kami juga menyediakan statistik dan visualisasi perbandingan hasil dan performa antar algoritma sehingga pengguna dapat memilih algoritma yang paling sesuai dengan kebutuhan pencariannya. Selain itu, pengguna dapat memanfaatkan fitur pencarian berbasis Regular Expression untuk mendeteksi pola teks yang lebih kompleks dan terstruktur, seperti kombinasi kata dan angka, langsung dari konten halaman web yang aktif. Chrome extension yang kami implementasikan juga memiliki fitur blur dan OCR serta memiliki tombol aktivasi algoritma string matching Aho-Corasick dan Rabin karp

5.2. Saran

Secara keseluruhan, implementasi algoritma string matching pada Chrome Extension ini telah berjalan dengan baik dan mampu melakukan pencarian kata kunci secara efektif pada halaman web yang dapat diakses. Namun, terdapat beberapa keterbatasan yang perlu diperbaiki untuk meningkatkan kualitas dan cakupan sistem kedepannya.

Pertama, basis data kata kunci yang tersedia saat ini masih terbatas. Sistem memerlukan koleksi kata kunci yang lebih luas dan beragam agar pencarian dapat mencakup lebih banyak konteks dan domain. Dengan kata kunci yang lebih

komprehensif, akurasi pencocokan terhadap konten halaman web akan meningkat secara signifikan, terutama untuk kasus-kasus pencarian yang bersifat spesifik atau teknis.

Kedua, terkait fitur ekstraksi teks berbasis OCR, sistem saat ini menghadapi kendala pada sejumlah website tertentu yang menerapkan kebijakan keamanan ketat, seperti Content Security Policy (CSP) dan pembatasan akses DOM, sehingga ekstensi tidak dapat dijalankan secara optimal pada halaman-halaman tersebut. Diperlukan pendekatan yang lebih baik untuk melewati batasan-batasan ini, misalnya melalui pemanfaatan background service worker yang memproses konten secara terpisah dari konteks halaman, atau dengan mengeksplorasi teknik injeksi skrip yang lebih kompatibel dengan kebijakan keamanan browser modern.

LAMPIRAN

Tautan *Repository* Github:

https://github.com/aufafaza/Tubes3_kakeguruistimashou.git

Surat Pernyataan

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (Generative AI), melainkan hasil pemikiran dan analisis mandiri.



Aufa Rienaldifaza Ahmad



Yavie Azka Putra Araly



Muhammad Dzaky Atha F.

Tabel 5.1 Poin Pengerjaan Spesifikasi

No	Poin	Ya	Tidak
1	Extension berhasil di-build dan di-load tanpa kesalahan pada chromium browser dan dikembangkan dengan TypeScript	✓	

2	KMP dan Boyer-Moore diimplementasikan from scratch	✓	
3	Regex menghandle format <kata><angka> dan berbagai edge case	✓	
4	Pencarian KMP & BM membaca keyword.txt secara iteratif dan tidak menggunakan built-in search function atau library eksternal	✓	
5	Exact matching dan fuzzy matching berjalan benar	✓	
6	Elemen DOM terdeteksi diberi highlight dan terhapus saat rescanning	✓	
7	Tooltip muncul saat hover dengan informasi keyword, algoritma, kemunculan, dan waktu eksekusi	✓	
8	Popup menampilkan statistik realtime (total keyword, perbandingan, waktu eksekusi, jumlah match)	✓	
9	[Bonus] Membuat Video		✓
10	[Bonus] Implementasi Algoritma Aho-Corasick dan Rabin Karp	✓	
11	[Bonus] Implementasi Censorship / Blur Teks	✓	
12	[Bonus] Implementasi Optical Character Recognition pada Gambar	✓	

Tabel 5.2 Pembagian Tugas

Nama - NIM	Tugas
Aufa Rienaldifaza Ahmad - 13523027	- Implementasi algoritma KMP, BM, dan Regex *matching* - Implementasi OCR - Integrasi algoritma dengan *chrome extension* - Laporan pada bagian langkah pemecahan masalah, dasar teori algoritma, pseudocode algoritma, dan snapshot implementasi.

Yavie Azka Putra Araly - 13524077	- Implementasi Algoritma Fuzzy Matching, Aho-Corasick, dan Rabin Karp - Integrasi algoritma dengan *chrome extension* - Membuat file HTML untuk testing - Laporan pada bagian langkah pemecahan masalah, proses pemetaan masalah, dan hasil & analisis pengujian
Muhammad Dzaky Atha F. - 18223124	- Implementasi Chrome Extension: popup, tooltip, fitur blur dan highlight - Laporan pada bagian mengenai chrome extension

REFERENSI

Chrome Developers.“Extensions.” developer.chrome.com/docs/extensions/.

Cox, Russ. “Regular Expression Matching Can Be Simple and Fast.” *Swtch.com*, 2022, swtch.com/~rsc/regexp/regexp1.html.

Rinaldi, Munir. *Pencocokan String (String/Pattern Matching)*. Diakses melalui: https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/23-Pencocokan-string-(2026).pdf

Rinaldi, Munir. *Pencocokan String* dengan Regex. Diakses melalui: https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2025-2026/24-String-Matching-dengan-Regex-(2026).pdf

Sinan Mp. “Complete Chrome Extension Tutorial (Manifest V3): Build & Publish a Real Extension.” *DEV Community*, 27 Feb. 2026, dev.to/sinan0333/complete-chrome-extension-tutorial-manifest-v3-build-publish-a-real-extension-4aeo. Accessed 30 May 2026.

Team, Groovy Web. “Groovy Web.” *Groovy Web*, 22 Feb. 2026, www.groovyweb.co/blog/chrome-extension-development-guide-2026. Accessed 30 May 2026.

Tijs Luitse. “Linear Chrome Extension for Developers — Honest Setup (2026).” *Operations*, 12 May 2026, operations.studionope.nl/blog/linear-per-project-workflow. Accessed 30 May 2026.